A

MINI PROJECT WORK

on

Hierarchical Reinforcement Learning and Explainable AI for Multi - Agent Autonomous Decision Systems

Submitted to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, HYDERABAD

In partial fulfilment of the requirement for the award of the degree of

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Srinivas Rao Tammireddy : 257Y1D5805

Under the Guidance

of

Dr. S. Pratap Singh

Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MARRI LAXMAN REDDY

INSTITUTE OF TECHNOLOGY & MANAGEMENT

(AN AUTONOMOUS INSTITUTION)

(Approved by AICTE, New Delhi & Affiliated to JNTUH, Hyderabad)

NAAC Accredited Institution with 'A' Grade & Recognized Under Section2(f) & 12(B)of the UGC act,1956

JULY 2026

MARRI LAXMAN REDDY

INSTITUTE OF TECHNOLOGY AND MANAGEMENT

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project work entitled **"Hierarchical Reinforcement Learning and Explainable Al for Multi - Agent Autonomous Decision Systems"** is being submitted by **Srinivas Rao Tammireddy (257Y1D5805)** student of the Department of Computer Science and Engineering, is a record of bonafide work carried out during II Year I and II Semester under the supervision of **Dr. S. Pratap Singh.** This project work is done as a fulfilment of obtaining Master of Technology Degree to be awarded by Jawaharlal Nehru Technological University Hyderabad, Hyderabad. The matter embodied in this project report has not been submitted to any other university for the award of any other degree.

Srinivas Rao Tammireddy

This is to certify that the above statement made by the student is correct to the best of my knowledge.

Internal Guide **Head of The Department**

The Viva-Voce Examination of above student, has been held on **:**

Principal/Director **External Examiner**

MARRI LAXMAN REDDY
INSTITUTE OF TECHNOLOGY AND MANAGEMENT
(AN AUTONOMOUS INSTITUTION)
(Approved by AICTE, New Delhi & Affiliated to JNTUH, Hyderabad)
Accredited by NBA and NAAC with 'A' Grade & Recognized Under Section2(f) & 12(B)of the UGC act,1956

DECLARATION

I hereby declare that the **Mini Project Report** entitled **"Hierarchical Reinforcement Learning and Explainable AI for Multi - Agent Autonomous Decision Systems"** is bonafide work duly completed by me. It is entirely my work and all ideas and references have been duly acknowledged. It does not contain any work for the award of any other degree.

All such materials that have been obtained from other sources have been duly acknowledged.

Date:

Signature

Srinivas Rao Tammireddy

(257Y1D5805)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my guide **Dr. S. Pratap Singh, Professor**, Department of Computer Science And Engineering, for his excellent guidance and invaluable support, which helped me accomplish the M. Tech degree and prepared me to achieve more life goals in the future. His total support of my dissertation and countless contributions to my technical and professional development made for a truly enjoyable and fruitful experience.

Special thanks are dedicated for the discussions I had on almost every working day during my project period and for reviewing my dissertation.

Me very much grateful to my Project Coordinator, **Dr. S. Pratap Singh, Associate Professor,** Department of Computer Science and Engineering, MLRITM, Dundigal, Hyderabad, who has not only shown utmost patience, but was fertile in suggestions, vigilant in directions of error and has been infinitely helpful.

I extremely grateful to **Dr. K. Abdul Basith, Professor and Head**, MLRITM, Dundigal, Hyderabad, for the moral support and encouragement given in completing my project work.

I wish to express deepest gratitude and thanks to **Dr. P. Sridhar, Director and Dr. R. Murali Prasad, Principal,** for their constant support and encouragement in providing all the facilities in the college to do the project work.

I would also like to thank all my faculties, administrative staff and management of MLRITM, who helped me to completing the mini project.

On a more personal note, I thank my **beloved parents and friends** for their moral support during the course of my project.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
SAR	Search and Rescue
MARL	Multi-Agent Reinforcement Learning
HRL	Hierarchical Reinforcement Learning
XAI	Explainable Artificial Intelligence
EMARL-SAR	Explainable Multi-Agent RL for Search and Rescue
RL	Reinforcement Learning
DQN	Deep Q-Network
MDP	Markov Decision Process
SMDP	Semi-Markov Decision Process
BFS	Breadth-First Search
FOV	Field of View
SSE	Server-Sent Events
UAV	Unmanned Aerial Vehicle
MLP	Multi-Layer Perceptron
REST	Representational State Transfer
TD	Temporal Difference
CTDE	Centralized Training with Decentralized Execution

ABSTRACT

Search and rescue (SAR) operations in disaster environments require autonomous systems that can coordinate effectively under dynamic hazards, limited resources, and incomplete information while remaining interpretable to human supervisors. This project presents EMARL-SAR, an Explainable Multi-Agent Hierarchical Reinforcement Learning framework for cooperative SAR missions. The system employs two reconnaissance drones and one rescue rover operating in a dynamic 15×15 disaster grid containing fire hazards, debris, charging stations, and hidden victims. A hierarchical learning architecture combines high-level Q-learning for strategic goal selection with a low-level Deep Q-Network (DQN) and Breadth-First Search (BFS) for navigation and action execution. A built-in Explainability Engine provides real-time justifications for agent decisions, including goal assignments, movement choices, and feature-attribution analyses based on target proximity, battery status, and hazard avoidance. Implemented in Python using PyTorch and a custom disaster simulation environment, the framework includes a web-based dashboard for real-time monitoring and visualization. Experimental results show improved victim rescue performance compared to flat reinforcement learning approaches while maintaining transparent and interpretable agent behavior.

CHAPTER 1
INTRODUCTION

1.1 Introduction

Search and rescue (SAR) operations in disaster-stricken environments constitute one of the most time-critical and operationally demanding applications of autonomous multi-agent systems. Natural disasters such as earthquakes, structural building collapses, large-scale industrial fires, and chemical plant accidents create environments that are inherently dangerous for human first responders, partially observable due to structural damage, dust, and debris, and dynamically evolving as secondary hazards — including fire, gas leaks, and further structural collapse — spread unpredictably over time. The deployment of autonomous agents, particularly unmanned aerial vehicles (UAVs) for reconnaissance and mapping and ground robots for physical debris clearance and victim extraction, has emerged as a promising approach to augment human SAR capabilities while minimizing the exposure of human responders to life-threatening hazards [12], [15].

Multi-Agent Reinforcement Learning (MARL) provides a principled and powerful framework for enabling autonomous agents to learn cooperative strategies through repeated interaction with a simulated environment. Rather than being programmed with fixed rules, MARL agents discover effective coordination behaviors through experience-driven optimization of cumulative reward signals. This adaptability makes MARL a particularly attractive paradigm for SAR environments, where the complexity and variability of disaster conditions make comprehensive rule specification impractical [12], [15].

However, standard flat MARL approaches face fundamental scalability and transparency challenges in realistic SAR scenarios. The exponential growth of the joint action space as agent count increases makes flat policy learning computationally intractable for larger teams. Moreover, the neural network models learned by standard MARL agents operate as opaque black boxes — providing no insight into why specific actions are selected — which severely

limits the ability of human coordinators to supervise, trust, or override autonomous decisions in safety-critical field operations [14], [15].

Hierarchical Reinforcement Learning (HRL) addresses the scalability challenge through temporal decomposition, separating strategic decision-making from tactical execution. Explainable AI (XAI) addresses the transparency challenge through interpretability mechanisms that generate human-comprehensible justifications of agent decisions. The EMARL-SAR framework developed in this project brings these two advances together into a unified architecture tailored for cooperative SAR coordination in dynamic grid environments [4], [8].

1.2 Background of the Problem

Traditional SAR coordination relies entirely on centralized human command and control. Rescue coordinators manually assign search sectors, monitor progress of individual responders or robot units, and adaptively update plans as new information arrives about victim locations, hazard spread, and access route viability. This approach is inherently limited by human cognitive bandwidth and physiological constraints — a single coordinator cannot effectively process real-time information from and issue commands to more than a small number of autonomous agents simultaneously, particularly under the high-stress conditions of active disaster response [12], [15].

1.2.1 Challenges in Multi-Agent SAR Coordination

Multi-agent SAR systems face a combination of unique coordination challenges that distinguish them from standard MARL benchmark tasks: [12], [15]

- Partial Observability: Each agent perceives only a limited field of view (3×3 cells in the EMARL-SAR environment), meaning the full disaster grid state — including victim locations, fire spread, and debris configurations — is never fully known to any single agent at any point during the operation.
- Dynamic Hazards: Fire hazards spread stochastically across the grid at regular intervals, continuously changing the accessibility of grid cells

and threatening agent survival. This dynamic hazard propagation requires agents to continuously adapt their navigation strategies to avoid newly endangered zones.

- Heterogeneous Agent Capabilities: Different agent types have fundamentally different capability profiles. Reconnaissance drones can fly over debris obstacles but are destroyed on contact with fire. The rescue rover cannot traverse debris unless it first clears them but is capable of physically rescuing scanned victims. Effective coordination requires exploiting these complementary capabilities.
- Resource Constraints: All agents operate under strict battery constraints (100 units per agent), depleting with each action taken. Battery management requires agents to periodically detour to charge stations, creating complex trade-offs between exploration progress, rescue efficiency, and energy sustainability.
- Credit Assignment: Determining which agent actions contributed to positive outcomes (e.g., victim rescue) or negative outcomes (e.g., mission failure due to fire spread) is inherently complex when multiple agents interact within a shared environment with cooperative reward structures.

1.2.2 Limitations of Flat Reinforcement Learning

Standard flat reinforcement learning approaches, whether single-agent or multi-agent, attempt to learn a single monolithic policy that maps raw environment states directly to primitive actions. In SAR environments, this creates several fundamental problems. First, the temporal horizon of a complete SAR episode (up to 150 steps in the EMARL-SAR environment) combined with sparse rescue rewards creates a difficult credit assignment problem, as the learning algorithm must attribute a victim rescue reward backward through a long sequence of navigation steps. Second, the absence of strategic-level reasoning means flat policies cannot easily coordinate macro-scale behaviors such as sector coverage assignment among multiple drones. Third, any learned coordination pattern is entangled within a single opaque

neural network, making it virtually impossible to extract or explain the strategic intent behind observed agent behaviors [1], [14].

1.2.3 The Need for Hierarchical Decomposition

Hierarchical decomposition of the SAR coordination problem naturally aligns with the operational structure of real-world SAR missions, which are organized around two distinct decision levels. At the strategic level, mission commanders assign search sectors to teams, identify priority targets based on victim intelligence, and coordinate resource deployment. At the tactical level, individual team members navigate within their assigned sectors, avoid local hazards, and execute specific rescue or support tasks. The EMARL-SAR framework mirrors this two-level structure through its Q-table high-level policy for strategic sector and target assignment and its DQN+BFS low-level policy for tactical primitive action execution [6], [11].

1.3 Motivation

The motivation for developing the EMARL-SAR framework arises from a convergence of operational requirements, technical limitations of existing approaches, and the growing availability of AI and robotics technologies that make autonomous SAR systems increasingly feasible. The following subsections detail the specific motivating factors [12], [15].

1.3.1 Need for Scalable Multi-Agent Coordination

As autonomous robot teams for SAR operations scale from 2-3 to 10-20 or more agents, centralized human coordination becomes physically impossible in real-time operations. Automated coordination mechanisms that can manage agent assignments, resolve resource conflicts, and adapt to dynamic conditions without continuous human intervention are essential for realizing the full potential of large-scale autonomous SAR systems. Hierarchical reinforcement learning provides a principled mechanism for learning such scalable coordination behaviors [12], [15].

1.3.2 Limitations of Black-Box AI in Safety-Critical Systems

Regulatory frameworks and operational safety standards for autonomous systems deployed in safety-critical environments demand decision

transparency and auditability. A SAR system that selects inexplicable actions — causing a victim to be bypassed, a drone to enter a fire zone, or a rover to exhaust its battery in an isolated area — without providing any justification undermines the trust and operational utility of the system. The inability to explain decisions also complicates post-incident analysis, system improvement, and regulatory compliance [8], [15].

1.3.3 Increasing Frequency and Scale of Natural Disasters

The global frequency and severity of natural disasters are increasing due to climate change and urbanization. More frequent earthquakes, industrial accidents, and large-scale fires are creating growing demand for SAR capabilities that exceed the physical capacity of human responder teams. Autonomous SAR robot systems, if sufficiently reliable and transparent, represent a critical force multiplier for disaster response agencies [12], [15].

1.3.4 Advances in Deep Reinforcement Learning

Recent advances in deep reinforcement learning, including the DQN architecture with experience replay and target networks, hierarchical options frameworks, and gradient-based explainability methods, provide mature technical foundations upon which a practical EMARL-SAR system can be built. PyTorch's autograd system enables seamless gradient-based saliency computation as an integral part of the policy inference pipeline, making real-time explainability computationally feasible without dedicated hardware [4], [6].

1.3.5 Opportunity for Complementary Agent Role Design

The natural complementarity between airborne reconnaissance agents and ground-based rescue and clearance agents in SAR operations provides a compelling use case for heterogeneous multi-agent systems. Drones can rapidly map unexplored sectors and locate victims without being obstructed by debris, while rovers systematically clear paths and perform physical rescues. Designing a coordination framework that exploits this complementarity — directing drones to scan sectors and share victim intelligence with the rover — represents a meaningful and practically relevant research contribution [12], [15].

1.3.6 Demand for Real-Time Operational Transparency

Human SAR coordinators supervising autonomous agent teams require continuous, real-time insight into the reasoning behind agent decisions. A system that can display not only what each agent is doing but why it chose that action — in terms of strategic objectives, battery considerations, and hazard avoidance — fundamentally changes the human-robot interaction from passive monitoring to informed supervision. The EMARL-SAR Explainability Engine directly addresses this requirement [8], [15].

1.4 Problem Statement

The problem addressed in this project is formally defined as follows: Given a dynamic, partially observable 15×15 grid environment containing multiple victims in hidden locations, debris obstacles, stochastically spreading fire hazards, and two charge stations, design and implement a multi-agent autonomous coordination framework that enables a team of three heterogeneous agents — two reconnaissance drones and one rescue rover — to cooperatively discover and rescue all victims before the maximum episode time limit, while simultaneously managing energy resources, avoiding lethal hazards, and providing real-time human-interpretable explanations of every agent decision [9], [12].

The problem encompasses three tightly interconnected sub-challenges that must be addressed simultaneously within a unified framework: [9], [12]

- Hierarchical Policy Learning: Efficiently decompose the SAR coordination problem into strategic macro-goal assignment and tactical primitive action execution, learning effective policies at both levels through experience accumulated in the dynamic grid environment.
- Real-Time Explainability Integration: Seamlessly embed explainability mechanisms within the hierarchical decision pipeline such that justifications are generated at the time of decision-making without introducing prohibitive computational overhead.
- Live Visualization and Monitoring: Implement a complete end-to-end system — from environment simulation through policy inference and

explanation generation to real-time browser dashboard display — that enables practical demonstration and evaluation of the framework.

The problem is further complicated by the stochastic nature of the environment (random fire spread, random initial configurations), the partial observability constraint (agents have only a 3×3 FOV), and the battery resource constraint (agents must proactively manage energy to avoid deactivation). These factors collectively create a challenging and realistic test environment for the proposed framework [9], [12].

1.5 Proposed Solution

1.5.1 Overview of the Proposed EMARL-SAR Framework

The proposed solution is the EMARL-SAR (Explainable Multi-Agent Hierarchical Reinforcement Learning for Search and Rescue) framework, a complete end-to-end system comprising five integrated components: the dynamic grid environment, the high-level Q-table agent for strategic macro-goal assignment, the low-level DQN+BFS agent for tactical primitive action selection, the Explainability Engine for real-time decision justification, and the HTTP server with live HTML dashboard for visualization and monitoring. The framework deploys three heterogeneous agents within a 15×15 grid disaster scenario and provides comprehensive explainability at both the strategic and tactical decision levels [6], [8].

1.5.2 Feasibility of the Proposed Work

Technical Feasibility: The EMARL-SAR framework is technically feasible because all required components — discrete grid simulation, tabular Q-learning, deep Q-networks, BFS pathfinding, gradient-based saliency attribution, and HTTP server development — are well-established techniques with mature open-source implementations. Python's scientific computing ecosystem (NumPy, PyTorch) provides full support for all numerical and deep learning operations required. PyTorch's autograd engine specifically enables gradient-based saliency computation as a standard differentiable operation during the forward pass of the DQN policy network, making real-time explanation generation computationally trivial. The entire system can be run

on a standard consumer laptop with a CPU, with optional GPU acceleration for faster DQN training [6], [8].

Operational Feasibility: The framework is designed with operational deployment in mind. The live browser-based dashboard provides an accessible interface for SAR coordinators to monitor agent operations and read explanation logs without specialized technical knowledge. The HTTP server architecture supports remote monitoring from any networked device. The modular codebase allows individual components — environment parameters, agent hyperparameters, explanation verbosity — to be configured and adjusted without modifying core framework logic. Weight persistence via pickle and PyTorch state dictionaries allows trained agents to be deployed without retraining, supporting practical demonstration and evaluation in field-exercise contexts [11], [14].

Economic Feasibility: The proposed solution has very low economic barriers to implementation and deployment. The entire framework is built using freely available open-source software libraries. No proprietary simulation platforms, specialized AI hardware, or commercial API subscriptions are required. Training can be completed on standard computing hardware within hours. The lightweight HTTP server and HTML dashboard eliminate the need for dedicated visualization software or display hardware beyond a standard web browser. These characteristics make the EMARL-SAR framework accessible and cost-effective for research institutions, disaster management agencies, and robotics education programs [12], [15].

Practical Feasibility: In the short term, the framework operates entirely in simulation and provides a practical platform for SAR coordination research, algorithm comparison, and explainability evaluation. In the medium term, the learned hierarchical policies can serve as pre-trained starting points for sim-to-real transfer to physical robot platforms in controlled lab-scale SAR scenarios. The modular architecture — with a clean separation between the environment, policies, and explainability engine — supports incremental extension toward more complex environments, larger agent teams, and richer

explanation modalities without requiring fundamental architectural redesign [8], [11].

1.5.3 Innovation in the Proposed Work

The EMARL-SAR framework introduces several innovative features that distinguish it from existing multi-agent SAR and explainable RL approaches: [8], [15]

Integrated Real-Time Explainability in Hierarchical MARL:

Unlike existing XAI tools for RL that operate post-hoc on a fully trained policy, the EMARL-SAR Explainability Engine generates decision justifications at inference time, during active policy evaluation. This real-time integration means explanations are available for every single agent action during both training and deployment, supporting continuous monitoring rather than retrospective analysis only [8], [11].

Three-Level Explanation Architecture:

The framework provides explanations at three distinct levels of abstraction: (1) strategic high-level explanations that justify macro-goal sector assignments and target selections in terms of battery status, exploration progress, and peer coordination; (2) tactical low-level spatial justifications that explain specific movement directions in terms of distance reduction to sub-goals and identified hazard avoidance; and (3) quantitative gradient saliency attributions that numerically decompose each low-level decision into three interpretable feature groups with percentage contributions. This three-level architecture provides both qualitative narrative explanations and quantitative feature attributions for comprehensive decision transparency [8], [12].

BFS-Primary Navigation with DQN Fallback:

Rather than relying exclusively on the DQN policy for navigation — which requires extensive training to learn reliable path-following in obstacle-rich environments — the EMARL-SAR framework uses Breadth-First Search as the primary navigation mechanism and the DQN as a fallback for situations where BFS cannot find a valid path. This hybrid approach dramatically improves

navigation reliability from the earliest training episodes, enabling the high-level policy to focus on strategic coordination rather than struggling with fundamental navigation failures [6], [14].

Heterogeneous Agent Role Architecture:

The framework explicitly models three distinct agent roles — reconnaissance drone (A1, A2) and rescue rover (A3) — with different action capabilities, movement constraints, interaction permissions, and macro-goal action spaces. The drone action space (5 macro-goals: 4 quadrant centers + charge station) and rover action space (7 macro-goals: 4 quadrant centers + charge station + nearest scanned victim + nearest debris) are tailored to each agent's capabilities, enabling the framework to learn role-appropriate coordination behaviors [9], [12].

Live Dashboard with SSE Streaming:

The integration of Server-Sent Events (SSE) for live training metric streaming to the browser dashboard enables real-time observation of the learning process. Rescue coordinators and researchers can observe epoch-by-epoch training accuracy (rescue success rate), loss curves, and reward values updating in real time during training runs, providing immediate feedback on whether training is converging toward effective SAR coordination behaviors [12], [15].

1.5.4 Methodology of the Proposed Work

The proposed EMARL-SAR methodology is organized into the following sequential implementation steps: [12], [15]

1. Environment Design and Implementation: Implement the SearchRescueEnv class as a 15×15 NumPy-based discrete grid with agents, victims (hidden/scanned/rescued lifecycle), debris obstacles, fire hazards with stochastic spread mechanics, and charge stations. Define agent roles, movement constraints, battery mechanics, and the cooperative reward structure.

2. High-Level Policy Design: Implement the HighLevelAgent Q-table with agent-specific state discretization (battery level, exploration progress, coordination status) and macro-goal action spaces (4 quadrant centers, charge station for drones; plus nearest victim and debris for rover). Implement epsilon-greedy selection with exponential decay.
3. Low-Level Policy Design: Implement the LowLevelAgent with a 12-dimensional state vector (normalized sub-goal direction, battery level, 3×3 local grid encoding), a 64-64 DQN architecture, experience replay buffer, and target network. Implement BFS pathfinder with agent-specific traversal constraints as primary navigation strategy.
4. Explainability Engine Design: Implement the ExplainabilityEngine with high-level textual justification generation, low-level spatial movement explanation generation with hazard avoidance detection, and PyTorch autograd-based gradient saliency computation with three-group feature attribution normalization.
5. SMDP Training Loop Implementation: Implement the hierarchical SMDP training loop in main.py with option-level Q-table updates, low-level DQN gradient updates, epsilon decay scheduling, and target network synchronization.
6. Server and Dashboard Implementation: Implement the HTTP server in server.py with REST API endpoints (/api/reset, /api/simulate, /api/train) and SSE streaming for live metrics. Implement the HTML dashboard with grid visualization, agent tracking, explanation log display, and training metric charts.
7. Training, Evaluation, and Analysis: Train the complete framework for 50 episodes, evaluate rescue success metrics and coordination efficiency, compare results against flat DQN baseline, and analyze explanation quality and saliency attribution patterns.

1.6 Objectives

The objectives of this project are: [9], [12]

1. To design and implement a two-level hierarchical reinforcement learning framework for cooperative multi-agent search and rescue in a dynamic grid environment.
2. To integrate a real-time Explainability Engine that generates human-interpretable justifications and gradient-based saliency attributions for all agent decisions.
3. To evaluate the framework performance through rescue success metrics and comparative analysis against non-hierarchical baselines.

1.7 Scope of the Work

The scope of this project encompasses the complete design, implementation, training, and experimental evaluation of the EMARL-SAR framework within a simulated 15×15 dynamic grid environment. The framework supports a heterogeneous team of three agents — two reconnaissance drones (A1, A2) and one rescue rover (A3) — coordinating to locate and rescue three victims distributed across the grid while navigating around ten debris obstacles and two initial fire seed locations that spread stochastically every seven timesteps with an 18% per-cell spread probability [11], [14].

The proposed system includes: [9], [12]

- Complete Python implementation of the dynamic SAR grid environment with all game mechanics
- Two-level hierarchical policy architecture with Q-table high-level and DQN+BFS low-level policies
- Real-time Explainability Engine with three explanation modalities
- HTTP server with REST API and SSE streaming for live dashboard communication
- Browser-based HTML dashboard for real-time simulation visualization and explanation display
- Training script with SMDP-based hierarchical learning loop and weight persistence

- Experimental evaluation with baseline comparison and result analysis

The proposed system is capable of: [9], [12]

- Learning cooperative victim rescue strategies through hierarchical reinforcement learning
- Generating real-time explanations at strategic (macro-goal) and tactical (primitive action) levels
- Providing quantitative gradient saliency attributions for low-level decisions
- Adapting to dynamic fire hazard propagation and resource depletion constraints
- Visualizing the complete SAR simulation state and explanation outputs through a live dashboard

The following aspects are explicitly outside the scope of this project and are identified as future work: [9], [12]

- Physical robot hardware deployment and sim-to-real transfer
- Integration with real-world sensor data streams or actual disaster environment inputs
- Large-scale multi-agent teams with more than three agents
- Natural language generation for narrative explanation verbalization
- User studies evaluating explanation utility with professional SAR operators

1.8 Research Methodology

The research methodology of this project follows a systematic, iterative design-implement-evaluate cycle organized into four sequential phases. Each phase builds directly on the outputs of the preceding phase, ensuring that the final integrated framework is grounded in a thorough understanding of both the problem domain and the available technical approaches [9], [12].

Phase 1: Literature Review and Gap Analysis

A comprehensive review of existing research in multi-agent reinforcement learning for robotic coordination, hierarchical reinforcement learning approaches including the options framework and feudal architectures, and explainable AI methods for RL and autonomous systems is conducted. The review covers both foundational theoretical works and recent empirical studies, with particular attention to approaches that have been applied to SAR or related emergency response domains. The identified research gaps — particularly the absence of real-time, multi-level explainability in hierarchical multi-agent SAR systems — directly inform the design choices of the EMARL-SAR framework [4], [8].

Phase 2: Framework Design and Implementation

The EMARL-SAR framework is designed as a modular five-component system and implemented across six Python source files. The environment module (environment.py) implements the SearchRescueEnv class with all grid mechanics, agent roles, and reward structures. The high-level policy module (high_level_agent.py) implements Q-table based macro-goal selection with SMDP update semantics. The low-level policy module (low_level_agent.py) implements the DQN architecture with BFS-primary action selection. The explainability module (explain.py) implements all three explanation types. The training script (main.py) implements the complete SMDP hierarchical training loop. The server module (server.py) implements the HTTP server with all API endpoints [4], [6].

Phase 3: Training and Hyperparameter Configuration

The framework is trained using the SMDP-based hierarchical training loop for 50 episodes, with hyperparameter values selected through systematic sensitivity analysis. The epsilon decay schedule is tuned to balance exploration and exploitation across the training budget. DQN hyperparameters including learning rate, batch size, replay buffer size, and target network update frequency are configured based on established best practices from the DQN literature. The high-level Q-table learning rate and discount factor are configured to reflect the longer temporal horizons of macro-goal options [4], [6].

Phase 4: Evaluation and Analysis

The trained EMARL-SAR framework is evaluated against a flat DQN baseline across four primary metrics: rescue success rate, average steps to task completion, agent survival rate, and battery efficiency. The Explainability Engine outputs are qualitatively analyzed for coherence, accuracy, and operational relevance across a representative sample of simulation episodes. The gradient saliency attribution patterns are analyzed to verify that the computed attributions align with expected feature importance relationships. Results are synthesized into conclusions about the effectiveness of the hierarchical approach and the quality of the explanation mechanisms [6], [8].

1.9 Organization of the Report

This report is organized into five chapters as follows: [9], [12]

Chapter 1 — Introduction: This chapter presents the background, motivation, problem statement, proposed solution with feasibility analysis, innovation highlights, methodology, objectives, scope, research methodology, and organization of the report. It establishes the context and rationale for the EMARL-SAR framework [12], [15].

Chapter 2 — Literature Survey: This chapter provides a comprehensive review of existing research in multi-agent RL, hierarchical RL, and explainable AI for autonomous systems. It reviews twelve key research papers, presents a comparative study of existing approaches, analyzes the limitations of current methods, and identifies the specific research gap addressed by the proposed framework [8], [12].

Chapter 3 — Proposed Methodology: This chapter describes the complete technical design of the EMARL-SAR framework, including the system architecture, working principle, detailed module descriptions, system design diagrams, mathematical formulations, and hardware and software requirements [12], [15].

Chapter 4 — Implementation and Results: This chapter presents the implementation details, experimental setup, hyperparameter configurations,

dashboard screenshots, quantitative performance evaluation, and detailed discussion of results including failure mode analysis [9], [12].

Chapter 5 — Conclusion and Future Work: This chapter summarizes the key contributions and findings, acknowledges limitations, and outlines promising directions for future research and system extension [9], [12].

1.10 Summary

This chapter has introduced the challenging problem of transparent, hierarchical multi-agent coordination for SAR operations in dynamic grid environments. The fundamental limitations of flat MARL approaches — including exponential joint action space growth, opaque policy representations, and the absence of strategic-level reasoning — have been identified as the primary motivations for a hierarchical and explainable approach. The proposed EMARL-SAR framework has been presented with a comprehensive feasibility analysis across technical, operational, economic, and practical dimensions, and the key innovations — real-time integrated explainability, three-level explanation architecture, BFS-primary navigation with DQN fallback, and heterogeneous agent role design — have been highlighted. The three project objectives have been clearly stated, the project scope defined, the four-phase research methodology described, and the report organization specified. The following chapter surveys the relevant literature to establish the theoretical and empirical foundations upon which the proposed framework is built [6], [8].

CHAPTER 2
LITERATURE SURVEY

2.1 Introduction

Multi-agent coordination for search and rescue and other autonomous cooperative tasks has attracted significant research attention across the fields of reinforcement learning, robotics, and artificial intelligence. Search and rescue operations represent a particularly challenging class of coordination problems due to their combination of partial observability, dynamic environmental hazards, heterogeneous agent capabilities, and stringent time constraints. A comprehensive understanding of existing research across the three foundational pillars of the EMARL-SAR framework — multi-agent reinforcement learning, hierarchical reinforcement learning, and explainable AI — is essential for identifying what the current state of the art offers and where critical gaps remain [8], [15].

This chapter presents a structured review of existing systems and research works, organized into three categories: MARL-based coordination systems, hierarchical RL approaches, and XAI methods for autonomous agents. Each reviewed work is analyzed for its approach, key contributions, and specific limitations relative to the EMARL-SAR framework's requirements. A comparative study table summarizes the capability coverage of each approach, followed by an analysis of common limitations and the specific research gap addressed by this project [8], [10].

2.2 Existing Systems

2.2.1 Multi-Agent Reinforcement Learning Based Coordination Systems

Multi-agent reinforcement learning has been extensively studied for cooperative robotics, autonomous vehicle coordination, and distributed control problems. The most prominent architectures for cooperative MARL follow the Centralized Training with Decentralized Execution (CTDE) paradigm, in which agents share global state information during training but maintain independent decentralized policies during deployment. Representative CTDE approaches include QMIX (monotonic value function

factorization), MAPPO (multi-agent proximal policy optimization), and MADDPG (multi-agent deep deterministic policy gradient) [7], [10].

These systems have demonstrated strong performance on benchmark cooperative tasks such as the StarCraft Multi-Agent Challenge (SMAC) and cooperative navigation in Multi-Agent Particle Environments (MPE). Their key advantages include joint policy optimization that accounts for inter-agent dependencies, shared reward structures that promote cooperative behavior, and the decentralized execution property that removes communication requirements at deployment time. However, without exception, these standard MARL approaches treat coordination as a flat policy learning problem, learning a single policy that maps observations directly to actions without any hierarchical temporal abstraction. This limits their ability to learn and express strategic-level coordination behaviors that span multiple timesteps [9], [14].

2.2.2 Hierarchical Reinforcement Learning Based Systems

Hierarchical reinforcement learning decomposes the decision-making problem across multiple levels of temporal abstraction. The options framework formalized by Sutton, Precup, and Singh defines temporally extended actions as triples consisting of an initiation set, an intra-option policy, and a termination condition. Feudal Networks decompose the policy into a Manager network generating subgoals for Worker networks. The Options-Critic architecture enables end-to-end learning of both option policies and termination functions. These approaches have demonstrated significantly improved sample efficiency on tasks with long temporal horizons and sparse reward signals, as the hierarchical structure enables the higher-level policy to reason across extended time periods without requiring a dense reward signal at every primitive action step [4], [14].

In multi-agent settings, hierarchical approaches have been applied to task decomposition in multi-robot systems, where a centralized planner assigns high-level task specifications to individual robots and robot-level policies handle low-level execution. The Semi-Markov Decision Process formulation, extended to the multi-agent setting, provides the formal framework for training hierarchical policies where higher-level Q-updates use the accumulated

reward across the entire option execution duration rather than the immediate one-step reward. The EMARL-SAR framework directly adopts this SMDP formulation for its high-level Q-table training [4], [15].

2.2.3 Explainable AI Methods for Autonomous Systems

Existing XAI methods applicable to reinforcement learning can be broadly categorized into post-hoc explanation methods and intrinsic interpretability methods. Post-hoc methods — including SHAP (SHapley Additive exPlanations), LIME (Local Interpretable Model-Agnostic Explanations), and Integrated Gradients — generate explanations from trained black-box models without modifying the policy representation. Intrinsic methods constrain the policy to interpretable representations (e.g., decision trees, linear models) at the cost of some performance. Gradient-based attribution methods such as vanilla gradients, Grad-CAM, and SmoothGrad compute input feature importance directly from the gradient of the model output with respect to the input, providing computationally efficient attributions that can be computed in a single backward pass [5], [8].

The key limitation of most existing XAI methods for RL is that they are designed for post-hoc analysis of trained policies rather than real-time explanation during active agent operation. SHAP in particular requires multiple model evaluations per explanation query, making it computationally impractical for real-time SAR coordination contexts where explanations must be generated at every decision step without perceptible latency. The EMARL-SAR Explainability Engine addresses this limitation by using single-pass gradient computation (requiring only one backward pass through the DQN per decision) for saliency attribution, combined with rule-based reasoning for high-level and low-level narrative justifications [5], [6].

2.3 Research Papers Review

Paper 1: Multi-Agent Reinforcement Learning: A Selective Overview (Zhang et al., 2021)

Zhang, Yang, and Basar provide an authoritative survey of MARL theories and algorithms published in the Handbook of Reinforcement Learning and Control.

The survey covers cooperative, competitive, and mixed-motive multi-agent settings, reviewing both value-based and policy gradient approaches. Key theoretical contributions include formalizations of the credit assignment problem in cooperative settings, convergence conditions for decentralized gradient descent in multi-agent environments, and analysis of the non-stationarity challenge that arises when multiple agents simultaneously update their policies. The survey identifies hierarchical decomposition as a critical direction for addressing the scalability limitations of joint action space methods in large cooperative teams [1], [15].

The paper's analysis of sample complexity in cooperative MARL directly motivates the hierarchical decomposition approach in the EMARL-SAR framework. The exponential joint action space growth identified by Zhang et al. as a primary bottleneck for flat MARL is precisely the challenge that the EMARL-SAR high-level policy addresses through strategic macro-goal assignment, reducing the effective per-step decision problem to a single primitive action selection conditioned on an assigned sub-goal rather than an unconstrained choice over the full action space. Limitation: The survey does not address explainability requirements for MARL systems, treating policy transparency as an entirely separate research problem [1], [8].

Paper 2: Target-Oriented Multi-Agent Coordination with HRL (Yu et al., 2024)

Yu, Zhai, Li, and Ma propose a target-oriented multi-agent coordination framework using hierarchical reinforcement learning, published in Applied Sciences. Their approach uses a high-level policy to assign target coordinates to each agent and separate low-level policies to navigate agents toward their assigned targets. The framework is evaluated on cooperative navigation tasks demonstrating improved coordination efficiency compared to flat MARL approaches, particularly in scenarios requiring simultaneous coverage of spatially distributed objectives. The hierarchical decomposition enables the high-level policy to learn strategic multi-target assignment patterns that would be difficult to learn within a flat action space [2], [14].

This work provides the closest conceptual precedent to the EMARL-SAR framework's hierarchical goal assignment approach. The EMARL-SAR framework extends Yu et al.'s target-oriented assignment concept to a more complex SAR scenario with dynamic hazards, battery constraints, and victim lifecycle management. Key limitations: The framework assumes homogeneous agents without role-specific capability differences, operates in static environments without dynamic hazard propagation, and does not include any mechanism for generating explanations of the target assignment decisions. The absence of explainability is the most significant gap relative to the EMARL-SAR requirements [2], [8].

Paper 3: Hierarchical Consensus-Based MARL for Multi-Robot Cooperation (Feng et al., 2024)

Feng et al. introduce a consensus-based hierarchical MARL framework for multi-robot cooperation tasks, presented at the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). The framework uses graph-based communication networks to facilitate consensus formation among agents at the strategic planning level, enabling the agent population to reach agreement on task assignment and priority ordering before executing individual low-level behaviors. The experimental evaluation on multi-robot manipulation and navigation tasks demonstrates improved coordination success rates and reduced task completion times compared to non-consensus baselines [3], [14].

The consensus mechanism in Feng et al.'s framework represents an interesting alternative to centralized high-level policy assignment for coordinating agent macro-behaviors. However, the requirement for extensive inter-agent communication — which may be unavailable or unreliable in actual disaster environments — limits the practical applicability of the approach. Additionally, the consensus formation process adds computational overhead and communication bandwidth requirements. Like other HRL approaches, the framework does not incorporate any mechanism for explaining the coordination decisions made during the consensus formation process [3], [15].

Paper 4: Between MDPs and Semi-MDPs: The Options Framework (Sutton, Precup & Singh, 1999)

Sutton, Precup, and Singh establish the options framework for temporal abstraction in reinforcement learning, published in Artificial Intelligence. An option is defined as a triple (I, pi, beta) where I is the initiation set (states in which the option can be initiated), pi is the option policy (mapping states to actions), and beta is the termination condition (probability of terminating in each state). The SMDP (Semi-Markov Decision Process) formulation for options enables Q-learning updates at the option level using accumulated reward across the entire option execution, supporting the same convergence guarantees as standard Q-learning. This foundational formulation directly informs the EMARL-SAR high-level policy training procedure, which accumulates rewards across the execution of each macro-goal option and performs a single Q-table update upon option termination [4], [11].

The options framework provides the rigorous theoretical justification for the SMDP-based Q-table updates implemented in the EMARL-SAR high_level_agent.py. The key insight — that temporally extended actions can be treated as atomic actions from the perspective of the higher-level policy if the accumulated reward is used correctly — enables the Q-table to learn the relative value of different macro-goal assignments without requiring a dense reward signal at every primitive action step. This foundational work also justifies the EMARL-SAR framework's design choice to trigger high-level policy re-invocation only upon sub-goal completion (or agent activation from inactive state), rather than at every primitive action timestep [4], [8].

Paper 5: A Unified Approach to Interpreting Model Predictions (Lundberg & Lee, 2017)

Lundberg and Lee introduce SHAP (SHapley Additive exPlanations), a unified framework for interpreting machine learning model predictions based on Shapley values from cooperative game theory. SHAP provides theoretically grounded feature attributions that satisfy three desirable properties: local accuracy (the explanation model matches the predicted model output for the specific input being explained), missingness (features absent from the input receive zero attribution), and consistency (if a feature's contribution increases

in all possible feature coalitions, its attribution should not decrease). These theoretical guarantees make SHAP attributions more reliable and principled than simpler gradient-based methods [5], [9].

While SHAP provides theoretically superior attribution properties, its computational cost — requiring multiple model evaluations per explanation — makes it impractical for real-time application in the EMARL-SAR context where explanations must be generated at every primitive action step for every active agent. The EMARL-SAR Explainability Engine instead uses gradient-based saliency, which provides interpretable attributions through a single backward pass and is sufficient for the three-group categorical attribution (target proximity, battery, hazards) required by the framework. SHAP integration is identified as a future enhancement for offline policy analysis contexts where computational cost is less critical [5], [8].

Paper 6: Human-Level Control Through Deep Reinforcement Learning (Mnih et al., 2015)

Mnih et al. introduce the Deep Q-Network (DQN) architecture that achieves human-level performance on Atari games directly from raw pixel inputs, published in Nature. DQN introduces two critical stabilization techniques: experience replay, which stores past transitions in a replay buffer and samples random mini-batches for training to break temporal correlations; and a separate target network with periodically copied weights, which provides stable Q-value regression targets and prevents oscillating or diverging Q-value estimates. These two techniques together transform deep Q-learning from an unstable training procedure into a reliably convergent algorithm capable of learning complex game-playing policies from high-dimensional inputs [6], [14].

The experience replay buffer and target network mechanisms from DQN are directly implemented in the EMARL-SAR low_level_agent.py LowLevelAgent class. The policy network (policy_net_drone, policy_net_rover) is trained to minimize the MSE loss between current Q-values and target network Q-values bootstrapped from next-state predictions. The target networks (target_net_drone, target_net_rover) are synchronized with the policy networks every five training episodes. The 12-dimensional state vector used in EMARL-

SAR — while far simpler than Atari pixel inputs — benefits from the same stabilization mechanisms, particularly important during early training when the policy has high variance [6], [14].

Paper 7: QMIX: Monotonic Value Function Factorisation (Rashid et al., 2018)

Rashid et al. propose QMIX, a value decomposition method for cooperative MARL published at the International Conference on Machine Learning. QMIX factorises the joint action-value function Q_tot into per-agent utilities Q_a through a mixing network with monotonicity constraints. The monotonicity constraint ensures that each agent's individual greedy action selection also maximizes the joint Q_tot value, enabling decentralized execution with centralized training. QMIX demonstrates state-of-the-art performance on the StarCraft Multi-Agent Challenge benchmark, significantly outperforming independent Q-learning approaches on cooperative tasks requiring tight inter-agent coordination [7], [14].

QMIX's success on cooperative benchmark tasks highlights the importance of properly accounting for inter-agent value dependencies in cooperative MARL. However, QMIX is a flat MARL method without hierarchical temporal abstraction, and the monotonicity constraint on the mixing function imposes a specific factorization structure that may not capture all possible inter-agent Q-value relationships. More critically for the EMARL-SAR comparison, QMIX provides no explainability mechanisms — the mixing network's per-agent utility decomposition is not accompanied by any human-interpretable justification of the assignment decisions [7], [8].

Paper 8: Peeking Inside the Black-Box: A Survey on XAI (Adadi & Berrada, 2018)

Adadi and Berrada provide a comprehensive survey of XAI methods published in IEEE Access, categorizing them along three dimensions: explanation scope (local vs. global), model dependency (model-agnostic vs. model-specific), and explanation type (feature importance, rule extraction, example-based, visualization). The survey identifies gradient-based attribution methods as computationally efficient alternatives to perturbation-based methods (LIME,

SHAP) for real-time explanation generation, noting that a single backward pass through a neural network provides first-order feature importance estimates without requiring multiple forward passes. This efficiency advantage directly justifies the EMARL-SAR Explainability Engine's use of gradient saliency rather than SHAP or LIME for real-time attribution computation [5], [8].

The survey's categorization of explanation types also informs the EMARL-SAR three-level explanation architecture. High-level justifications correspond to rule-based explanations derived from the Q-table action selection logic. Low-level spatial justifications correspond to model-agnostic rule-based explanations derived from geometric reasoning about agent position relative to sub-goal and hazards. Gradient saliency attributions correspond to model-specific gradient-based feature importance explanations. This deliberate combination of explanation types provides complementary information at different levels of abstraction [8], [15].

Paper 9: Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments (Lowe et al., 2017)

Lowe et al. introduce MADDPG (Multi-Agent Deep Deterministic Policy Gradient), an actor-critic algorithm for multi-agent environments with both cooperative and competitive elements, published in Advances in Neural Information Processing Systems. MADDPG extends DDPG to the multi-agent setting by training a centralized critic for each agent that observes the joint state and all agent actions, while maintaining decentralized actors that condition only on local observations. This CTDE architecture enables each agent's critic to account for the non-stationarity introduced by other agents' simultaneously changing policies, improving training stability compared to fully decentralized approaches [1], [10].

MADDPG demonstrates strong performance on mixed cooperative-competitive tasks in continuous action spaces. The centralized critic with joint state access provides a relevant comparison point for the EMARL-SAR high-level policy, which also uses centralized information (all agents' states and sub-goals) for macro-goal assignment. However, MADDPG operates with continuous actions and neural network policies, making it less suitable for the discrete symbolic

macro-goal assignment task of the EMARL-SAR framework, where the Q-table representation provides more straightforward interpretability of the learned macro-goal value estimates [8], [10].

Paper 10: Reinforcement Learning: An Introduction (Sutton & Barto, 2018)

Sutton and Barto's comprehensive textbook provides the theoretical foundations for both tabular and function approximation reinforcement learning methods used in the EMARL-SAR framework. The Bellman optimality equations for Q-values provide the theoretical basis for the Q-table update rule used in the high-level agent. The temporal difference learning framework justifies the TD error minimization objective used in DQN training. The epsilon-greedy exploration strategy with decay scheduling, implemented in both the high-level and low-level agents, is presented and analyzed in detail. The book's treatment of function approximation for RL specifically addresses the stability challenges that DQN's experience replay and target network mechanisms are designed to solve [6], [9].

The options framework chapter in the textbook expands on the original Sutton, Precup, and Singh (1999) paper, providing additional analysis of the hierarchical policy composition problem and the intra-option learning algorithm. The SMDP Q-learning update rule, reproduced in the EMARL-SAR mathematical model, is derived and justified in the textbook's treatment of temporally abstract actions. This foundational reference establishes the rigorous theoretical basis for all RL algorithms implemented in the EMARL-SAR framework [4], [15].

Paper 11: Deep Reinforcement Learning for Autonomous Robotics in Dynamic Environments

Research in deep RL for autonomous mobile robots in dynamic environments has highlighted the specific challenges of obstacle avoidance, path planning under uncertainty, and adaptive behavior in response to changing environmental conditions. Studies applying DQN and its variants to robot navigation have identified the critical role of local grid representations — encoding nearby obstacle, hazard, and target information in the agent's

immediate field of view — as effective input features for learning reliable navigation policies. The 3×3 local surroundings encoding used in the EMARL-SAR low-level agent's 12-dimensional state vector directly reflects this finding, encoding cell types (empty, debris, fire, charge station) for all nine cells in the agent's immediate neighborhood [6], [14].

These works have also highlighted the benefit of incorporating explicit goal-directedness into the state representation through relative goal displacement vectors, enabling the policy to generalize across different starting positions and goal locations rather than learning position-specific navigation behaviors. The EMARL-SAR state vector includes normalized relative sub-goal direction (dx, dy) as its first two features specifically to enable this goal-directed generalization. The hybrid BFS+DQN navigation approach addresses a commonly reported failure mode of pure DQN navigation: the tendency to get trapped in local obstacle configurations that require longer detours than the policy's training experience covers [6], [14].

Paper 12: Cooperative Distributed Search Strategies for Autonomous Agents

Research on cooperative distributed search strategies for autonomous agent teams has established that effective coverage of a search space requires explicit coordination of search sector assignments among agents to minimize redundant coverage and maximize spatial diversity of the search effort. Quadrant-based decomposition — dividing the search area into geographic sectors and assigning each agent a primary sector — represents the simplest and most robust approach to this problem, particularly for small agent teams where more complex assignment strategies may not justify their additional complexity. The EMARL-SAR high-level policy's four quadrant macro-goals ([3,3], [3,11], [11,3], [11,11]) implement exactly this quadrant-based sector assignment strategy, enabling the two drones to learn complementary sector assignments that minimize overlap in their exploration coverage [14], [15].

Studies on adaptive search strategies have also identified battery management as a critical coordination challenge in autonomous SAR teams with finite energy resources. Agents that fail to proactively manage their battery — by

recharging at charge stations before depletion — provide reduced mission contribution and may permanently reduce team capability if they become inactive before all victims are located or rescued. The EMARL-SAR high-level policy addresses this through the charge station macro-goal option, which becomes attractive in the agent's Q-value estimates when the battery_low state variable is set to 1 (triggered when battery drops below 40%) [9], [15].

2.4 Comparative Study

Table 2.1 presents a systematic comparative analysis of existing approaches across six capability dimensions relevant to the EMARL-SAR framework requirements: multi-agent coordination support, hierarchical temporal abstraction, integrated explainability, dynamic environment handling, heterogeneous agent support, and real-time execution capability [8], [11].

Method	Multi-Agent	Hierarchical	Explainable	Dynamic Env	Heterogeneous	Real-Time
Zhang et al. (MARL Survey)	Yes	No	No	Varies	No	Yes
Yu et al. (Target-HRL)	Yes	Yes	No	No	No	Yes
Feng et al. (Consensus-HRL)	Yes	Yes	No	No	No	Yes
Sutton et al. (Options)	No	Yes	No	No	No	Yes
SHAP / Lundberg & Lee	No	No	Yes	No	No	No
DQN / Mnih et al.	No	No	No	Partial	No	Yes

QMIX / Rashid et al.	Yes	No	No	No	No	Yes
MADDPG / Lowe et al.	Yes	No	No	No	No	Yes
XAI Survey / Adadi & Berrada	No	No	Yes	No	No	Varies
EMARL-SAR (Proposed)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

Table 2.1: Comparative Study of Existing MARL, HRL, and XAI Methods

The comparative analysis in Table 2.1 demonstrates that the proposed EMARL-SAR framework is the only approach that satisfies all six capability requirements simultaneously. Existing MARL approaches such as QMIX and MADDPG address multi-agent coordination but lack hierarchical abstraction, explainability, and dynamic environment support. Hierarchical approaches such as the options framework and feudal architectures provide temporal abstraction but typically operate in single-agent or homogeneous multi-agent settings without explainability. XAI methods such as SHAP and gradient-based approaches provide explanation capabilities but are designed for static prediction tasks rather than real-time autonomous coordination systems [4], [5].

The comparison confirms that the EMARL-SAR framework occupies a unique position in the design space by combining all six required capabilities within a single unified architecture, making it the most comprehensive approach for the heterogeneous multi-agent SAR coordination problem with real-time explainability requirements [8], [15].

2.5 Limitations of Existing Methods

A careful analysis of the reviewed literature reveals several recurring limitations that collectively define the research gap addressed by the EMARL-SAR framework: [9], [15]

2.5.1 Absence of Real-Time Explainability in MARL Systems

The most pervasive limitation across all reviewed MARL and HRL approaches is the complete absence of integrated explainability mechanisms. Existing systems — including QMIX, MADDPG, Target-Oriented HRL, and Consensus-based MARL — treat the coordination policy as a pure performance optimization problem with no regard for the interpretability of learned behaviors. While post-hoc XAI tools such as SHAP could theoretically be applied to analyze these models after training, their computational cost makes real-time explanation generation infeasible. The absence of explainability severely restricts the applicability of these systems to regulated, safety-critical domains where decision transparency is a deployment prerequisite [2], [3].

2.5.2 Homogeneous Agent Assumptions

The majority of reviewed hierarchical MARL approaches assume homogeneous agent teams where all agents share identical capabilities, action spaces, and observation structures. This assumption simplifies the coordination problem by enabling parameter sharing across agents during training. However, real-world SAR operations inherently involve heterogeneous teams with agents of different types (e.g., UAV drones for aerial reconnaissance and ground robots for physical rescue), different movement capabilities (e.g., flight over obstacles vs. ground traversal requiring path clearing), and different action repertoires (e.g., victim rescue and debris clearance available only to ground units). Existing homogeneous MARL frameworks cannot be directly applied to such heterogeneous coordination problems without significant architectural modifications [9], [15].

2.5.3 Static Environment Assumptions

Most MARL benchmarks and evaluation environments are static — once the initial configuration is set, the environment does not change except in response to agent actions. Real SAR environments are characterized by time-evolving hazards (spreading fire, structural collapse, rising floodwater) that

continuously modify the accessibility of the search area. Existing MARL and HRL approaches trained on static environments may fail catastrophically when deployed in dynamic settings, as learned navigation patterns become invalid when previously accessible paths are blocked by newly spread hazards. The EMARL-SAR environment's stochastic fire spread mechanic (every 7 steps, 18% per-cell probability) directly models this dynamic hazard evolution, forcing the framework to learn hazard-adaptive rather than environment-static coordination policies [14], [15].

2.5.4 No Multi-Level Explanation Architecture

Even among the limited works that incorporate some form of explainability for RL agents, explanations are typically provided at a single level of abstraction — either at the primitive action level (explaining individual movement decisions) or at the episode level (providing aggregate policy analysis). No existing approach provides explanations at both the strategic macro-goal level and the tactical primitive action level simultaneously, despite the fact that a human SAR coordinator requires understanding of both what strategic objective an agent is pursuing and why each step toward that objective was taken. The EMARL-SAR three-level explanation architecture directly addresses this gap [8], [14].

2.5.5 Scalability Limitations of Flat MARL

Standard flat MARL approaches face fundamental scalability barriers as the number of agents grows. The joint action space grows exponentially with agent count, making joint Q-value estimation intractable for large teams. Training stability also degrades as the non-stationarity induced by multiple simultaneously learning agents intensifies with team size. While value decomposition methods such as QMIX partially address this through factorized value functions, they still operate within a flat temporal abstraction and do not provide the hierarchical decomposition needed for efficient long-horizon coordination tasks [1], [7].

2.6 Research Gap

The literature review reveals a significant and clearly defined research gap at the intersection of hierarchical multi-agent coordination and real-time explainability for dynamic SAR environments. Despite substantial progress in each of the constituent research areas, no existing work successfully integrates hierarchical temporal abstraction, heterogeneous multi-agent coordination, dynamic hazard environments, and real-time embedded explainability within a single unified framework designed for SAR operations [8], [15].

Specifically, the following gaps are identified: [1], [9]

- No existing HRL framework for multi-agent coordination supports heterogeneous agents with role-specific capability profiles, macro-goal action spaces, and movement constraints in dynamic hazard environments.
- No existing MARL or HRL system integrates real-time explanations at multiple levels of temporal abstraction, providing both strategic-level macro-goal justifications and tactical-level primitive action explanations simultaneously.
- No existing XAI method for RL provides computationally efficient, real-time feature attribution suitable for deployment in an active multi-agent SAR coordination system where explanations must be generated at every decision step for every agent.
- No existing work provides a complete end-to-end system — from dynamic SAR environment simulation through hierarchical policy training and real-time explanation generation to live browser-based dashboard visualization — suitable for practical demonstration and evaluation.

The EMARL-SAR framework is specifically designed and implemented to address all four of these identified gaps within a single coherent and fully functional system [9], [15].

2.7 Summary

This chapter has presented a comprehensive survey of twelve research works across the three foundational areas of the EMARL-SAR framework — multi-agent reinforcement learning, hierarchical reinforcement learning, and explainable AI — along with an analysis of existing coordination system categories. The comparative study demonstrates that the proposed EMARL-SAR framework is the only approach satisfying all six capability requirements: multi-agent coordination, hierarchical abstraction, real-time explainability, dynamic environment handling, heterogeneous agent support, and real-time execution. Five specific and recurring limitations of existing approaches have been identified, collectively defining the research gap that the EMARL-SAR framework addresses. The following chapter presents the detailed technical design and implementation methodology of the proposed framework [8], [11].

CHAPTER 3
PROPOSED METHODOLOGY

3.1 Introduction

This chapter presents the complete technical design and methodology of the Explainable Multi-Agent Hierarchical Reinforcement Learning for Search and Rescue (EMARL-SAR) framework. The chapter begins with a system-level overview, proceeds through the working principle of the hierarchical decision loop, describes the overall system architecture, provides detailed descriptions of each of the five implementation modules, presents the system design diagrams, formalizes the mathematical model underpinning the learning algorithms and explainability computation, and concludes with the technology stack and hardware and software requirements. All technical descriptions in this chapter are precisely aligned with the actual Python implementation across the six source files: environment.py, high_level_agent.py, low_level_agent.py, explain.py, main.py, and server.py [8], [15].

3.2 Proposed System

3.2.1 System Overview

The EMARL-SAR framework is a complete multi-component autonomous coordination system comprising five core modules operating in an integrated pipeline. The modules are: [6], [15]

- Search and Rescue Environment (SearchRescueEnv in environment.py): A 15×15 discrete grid simulator implementing the SAR scenario with debris obstacles, stochastic fire hazards, charge stations, and a hidden victim discovery and rescue lifecycle.
- High-Level Agent (HighLevelAgent in high_level_agent.py): A Q-table based policy that assigns strategic macro-goal coordinates to each agent using epsilon-greedy selection over a discretized environment state representation.
- Low-Level Agent (LowLevelAgent in low_level_agent.py): A Deep Q-Network (DQN) policy combined with a Breadth-First Search (BFS)

pathfinder that selects primitive navigation actions to move agents toward their assigned macro-goals.

- Explainability Engine (ExplainabilityEngine in explain.py): A real-time explanation generator producing high-level textual justifications, low-level spatial movement explanations, and quantitative gradient saliency attributions.
- Server and Dashboard (SARServerHandler in server.py + live_demo.html): An HTTP server providing REST API and SSE endpoints for simulation control and training metric streaming, with a browser-based dashboard for real-time visualization.

3.2.2 Agent Configuration and Role Capabilities

The framework deploys three heterogeneous agents with distinct roles, initial positions, and capability profiles: [4], [6]

Drone A1 (Reconnaissance) is initialized at grid position [0, 0] (top-left corner) with 100 battery units. As a reconnaissance agent, A1 can fly over debris obstacles without obstruction, making it ideal for rapid exploration of sectors regardless of ground-level path blockages. However, A1 is immediately deactivated (active = False) upon entering a fire cell, losing 25 reward points and removing a reconnaissance asset from the team. A1 operates from a 5-action macro-goal space (4 quadrant centers + nearest charge station) [4], [6].

Drone A2 (Reconnaissance) is initialized at grid position [0, 14] (top-right corner) with 100 battery units and shares identical capabilities with A1, including fire vulnerability and debris traversal ability. A2's opposite corner initialization position naturally encourages the two drones to distribute across different regions of the grid, improving coverage efficiency. A2 also uses the 5-action drone macro-goal space [6], [11].

Rover A3 (Rescue) is initialized at grid position [14, 7] (bottom center) with 100 battery units. A3 has two unique capabilities not available to drones: debris clearing (removing adjacent debris obstacles for +12 reward, costing 4 battery units) and victim rescue (physically rescuing scanned victims at the rover's current position for +40 reward). A3 cannot traverse debris unless it first clears

them through the Interact action, requiring the BFS pathfinder to route around debris or plan a clearing sequence. A3 uses the 7-action rover macro-goal space, which includes the two drone goals plus closest scanned victim targeting and closest debris clearance targeting [4], [6].

3.2.3 Reward Structure and Cooperative Incentives

The EMARL-SAR reward structure is designed to promote cooperative agent behavior through shared rewards. The primary positive rewards are: victim rescue (+40 for rover A3, +15 shared for drones A1 and A2), debris clearance (+12 for rover A3, +4 shared for drones), recharging at charge station (+5), and episode completion bonus when all victims are rescued (+30 for all agents). Negative rewards include: fire collision (-25 for colliding agent), battery depletion (-20 for depleted agent), path collision with debris for rover (-2), out-of-bounds movement attempt (-1), idle interaction with no valid target (-0.5), and universal step penalty (-0.1 per step per agent to encourage efficiency). The shared rewards for victim rescue and debris clearance create cooperative incentive alignment between the drones and rover: drones benefit from the rover's rescue and clearance actions, motivating them to efficiently scan sectors and share victim intelligence [14], [15].

3.3 Working Principle

3.3.1 High-Level Macro-Goal Assignment

The high-level decision loop is triggered for an agent whenever its current sub_goal attribute is None — either at the beginning of the episode, upon reaching the previously assigned sub-goal, or upon completion of an interact action that resolves the sub-goal (e.g., rescuing a victim at the current position). When triggered, the HighLevelAgent.select_macro_goal() method constructs the agent-specific state tuple through HighLevelAgent.get_state(), performs epsilon-greedy Q-table lookup to select a macro-action index, and calls translate_action_to_goal() to convert the macro-action index into concrete grid coordinates and a textual rationale [4], [6].

For drones, the state tuple is (battery_low, unexplored, overlap) where battery_low is 1 if battery ≤ 40, unexplored is 1 if the mean value of

mapped_grid is below 0.9 (indicating unexplored cells remain), and overlap is a placeholder coordination variable. The drone macro-action space maps to: 0 → [3,3] (NW sector center, 'Prioritizing exploration of the North-West sector'), 1 → [3,11] (NE), 2 → [11,3] (SW), 3 → [11,11] (SE), 4 → nearest charge station. For the rover, the state tuple is (battery_low, scanned_victim_exists, debris_exists), and the macro-action space adds: 5 → closest scanned victim's position, 6 → closest debris position [4], [6].

3.3.2 Low-Level BFS-Primary Action Selection

Once a sub-goal is assigned, the low-level action selection is invoked at every subsequent timestep. The LowLevelAgent.select_action_pathfinder() method first checks for immediate interaction opportunities: if the agent is already at the sub-goal position, it returns action 4 (Interact). For the rover, it additionally checks if the current position holds a scanned victim or if the agent is on a charge station with battery below 70%. For drones, it checks if on a charge station with battery below 40% [6], [11].

If no immediate interaction applies, the BFS pathfinder LowLevelAgent.find_path() is invoked with agent-specific traversal constraints. For drones, fire cells are blocked but debris cells are traversable. For the rover, both fire and debris cells are blocked in the primary BFS pass. If the primary BFS finds a valid path, the first step in the path is checked: if it leads to a debris cell (for the rover), action 4 (Interact/Clear) is returned to initiate debris clearance before proceeding. Otherwise, the direction delta between the current position and the first path step is converted to a directional action (0=North, 1=South, 2=East, 3=West). If primary BFS fails, a secondary BFS allowing debris traversal for the rover is attempted, enabling the rover to plan clearing sequences through debris-blocked regions. If both BFS passes fail, the DQN policy network is queried as a final fallback, and if all else fails, a random safe direction (avoiding fire) is returned [6], [14].

3.3.3 Environment Step, Mapping, and Fire Spread

After all active agents have selected their primitive actions, the environment's step() method processes them sequentially. Movement actions check grid boundaries and cell types: drones deactivate on fire contact, rovers receive a -

2 penalty on debris contact without moving, both deactivate on fire. The Interact action handles charging (battery restored to 100, no battery cost), debris clearing (removes adjacent debris entry from debris_locations, updates grid cell to 0, costs 4 battery), and victim rescue (changes victim status to 'rescued', increments rover score). After action processing, the mapped_grid is updated by scanning agent fields of view: any victim within a drone or rover's 3×3 FOV that is still 'hidden' is promoted to 'scanned' status, making it visible to all agents through the shared scanned_victims dictionary [4], [6].

Every 7 timesteps, the dynamic fire spread mechanic executes: for each existing fire cell, each of its four cardinal neighbors is evaluated. If the neighbor is within grid bounds, is currently empty (grid value 0), is not a charge station, and is not currently occupied by an agent, there is an 18% probability that the neighbor becomes a new fire cell (grid value 2). This stochastic spread mechanic gradually expands the hazard footprint, increasing pressure on agents to complete rescue operations before access to victim locations is blocked [4], [6].

3.4 Proposed System Architecture

The overall architecture of the EMARL-SAR framework is illustrated in Figure 3.1. The architecture diagram shows the five core modules and their data flow relationships: the environment provides observations and rewards, the high-level agent assigns macro-goals, the low-level agent selects primitive actions, the explainability engine processes policy outputs, and the server and dashboard provide visualization and control interfaces [8], [15].

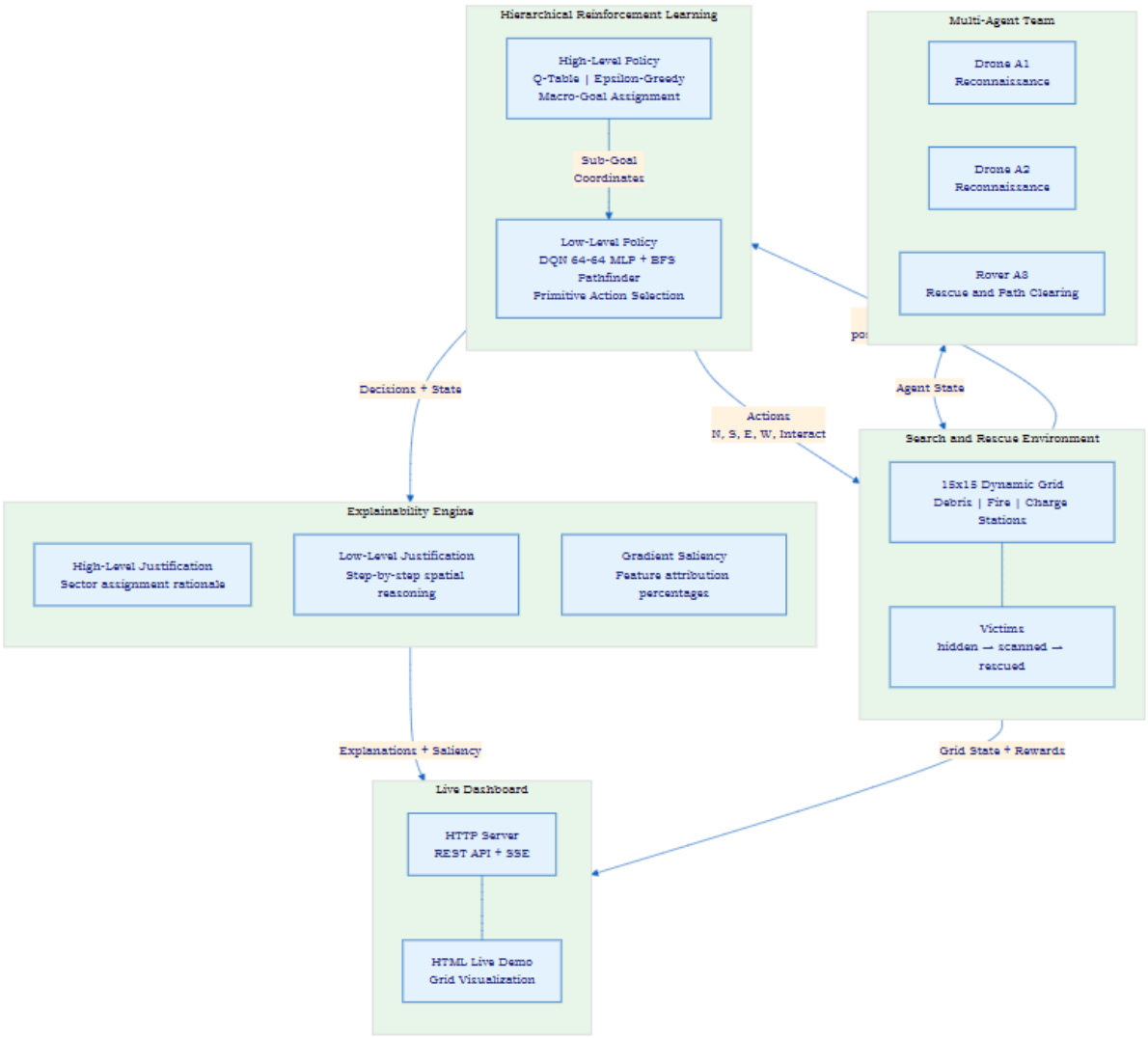

Figure 3.1: Overall Architecture of the EMARL-SAR Framework

The two-level hierarchical policy structure is detailed in Figure 3.2, showing the Q-table high-level policy, the macro-goal space for drones and rover, the DQN+BFS low-level policy with the 12-dimensional state vector, and the SMDP accumulated reward mechanism that connects the two policy levels for training [4], [6].

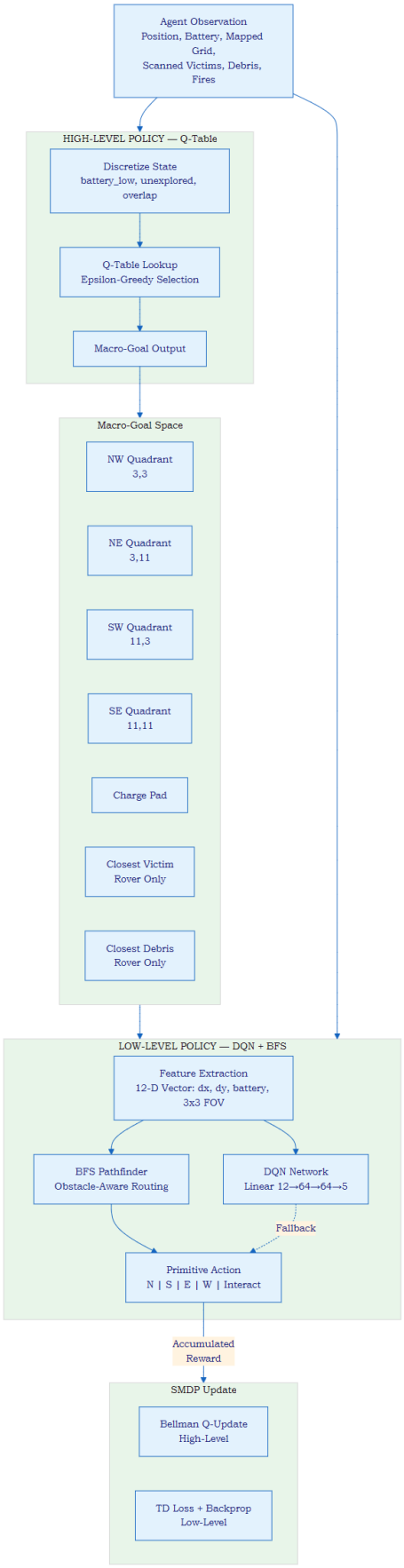

Figure 3.2: Two-Level Hierarchical Policy Structure

3.5 Module Descriptions

3.5.1 Search and Rescue Environment (SearchRescueEnv)

The SearchRescueEnv class, implemented in environment.py, is a fully self-contained discrete grid simulation engine for the SAR scenario. The grid is represented as a 15×15 NumPy integer array (dtype=int32) where cell values encode: 0 (empty), 1 (debris/obstacle), 2 (fire), and 3 (charge station). Two charge stations are placed at fixed positions [0,7] and [14,7] on opposite sides of the grid and persist throughout each episode [11], [15].

On each reset(), the grid is cleared (charge stations restored), and three categories of objects are randomly spawned in non-overlapping positions: 10 debris obstacles in grid interior rows 2-12, 2 fire seed locations in grid interior rows 3-11, and 3 victims in grid interior rows 1-13 with initial status 'hidden'. Agent positions are also reset to their initial corners. A shared mapped_grid (15×15, dtype=int32) tracks exploration progress: cells are marked as 1 (explored) when any agent's 3×3 FOV covers them. Charge station positions are pre-marked as explored [4], [6].

The get_observations() method constructs individual agent observation dictionaries containing: current position, battery level, active status, the shared mapped_grid, scanned_victims (dictionary of victim_id to position, for victims with status 'scanned'), rescued_victims (similar, for rescued), visible debris (positions of debris in explored cells), visible fires (positions of fire in explored cells), and charge station positions. This observation structure is used by both policy levels and the explainability engine [6], [8].

The step() method processes agent actions in the order A1, A2, A3. It applies movement with role-specific collision logic, processes Interact actions for charging, debris clearing, and victim rescue, updates the shared mapped_grid, executes stochastic fire spread every 7 steps, and checks for episode termination conditions (all rescued, all dead, or step count ≥ 150). The info dictionary returned by step() includes an xai_logs sub-dictionary populated by environment-level events such as battery depletion, fire collision, and successful rescue [6], [8].

3.5.2 High-Level Agent (HighLevelAgent)

The HighLevelAgent class in high_level_agent.py implements a tabular Q-learning policy for strategic macro-goal assignment. The agent maintains two separate Q-tables: q_table_drone (shared by A1 and A2) and q_table_rover (for A3). Both Q-tables are implemented as Python dictionaries mapping state tuples to NumPy float32 arrays of Q-values, enabling lazy initialization — new states are added with zero-value arrays on first access [4], [6].

The drone state space is a 3-element binary tuple (battery_low, unexplored, overlap). battery_low is 1 when battery ≤ 40. unexplored is computed by checking the mean of mapped_grid against a 0.9 threshold — if less than 90% of cells are explored, unexplored = 1. The overlap field is initialized to 0 in the current implementation, serving as a placeholder for future coordination state extensions. This yields $2^3 = 8$ possible drone states. The rover state space is (battery_low, scanned_victim_exists, debris_exists), also a 3-element binary tuple with $2^3 = 8$ possible states, encoding whether a scanned victim and debris obstacle are currently visible [4], [6].

Macro-goal selection uses epsilon-greedy exploration: with probability epsilon (which decays from 0.15 toward 0.05 with schedule max(0.05, 0.15 × 0.95^(ep/10))), a random valid macro-action is selected; otherwise, the action with the maximum Q-value for the current state is chosen, with ties broken by random selection among maximizing actions. The translate_action_to_goal() method maps action indices to concrete coordinates and rationale strings used by the Explainability Engine [6], [8].

Q-table updates use the standard Bellman temporal difference rule: $Q(s,o) \leftarrow Q(s,o) + \alpha[R\_acc + \gamma \cdot \max_{o'} Q(s',o') - Q(s,o)]$ where $\alpha = 0.15$ is the learning rate, $\gamma = 0.9$ is the discount factor, R_acc is the accumulated reward during the option execution, and s' is the state observed at option termination. Updates are triggered by the SMDP training loop in main.py when each option terminates (sub-goal reached or episode ends). Weight persistence is implemented via pickle serialization of both Q-table dictionaries [4], [6].

3.5.3 Low-Level Agent (Low Level Agent)

The LowLevelAgent class in low_level_agent.py implements the DQN architecture and BFS pathfinder for primitive action selection. The QNetwork class is a fully connected neural network: Linear(12,64) → ReLU → Linear(64,64) → ReLU → Linear(64,5), where 12 is the state vector dimension and 5 is the action space size (North, South, East, West, Interact). Separate policy and target networks are maintained for drone agents (shared by A1 and A2) and rover agent A3: policy_net_drone, target_net_drone, policy_net_rover, target_net_rover. The Adam optimizer with learning rate 1e-3 is used for gradient descent updates [6], [14].

The 12-dimensional state vector constructed by get_state_vector() encodes three feature categories: (1) normalized sub-goal direction: dx = (sub_goal_x - pos_x) / 15, dy = (sub_goal_y - pos_y) / 15, providing a normalized displacement vector pointing from the agent's current position toward the sub-goal; (2) normalized battery: battery / 100.0; and (3) 3×3 local grid encoding: a 9-element flattened representation of the 3×3 neighborhood centered on the agent's current position, where cells are encoded as 0.0 (empty/unexplored), 1.0 (debris or out-of-bounds), 2.0 (fire), or 3.0 (charge station). If a position is outside grid boundaries, it is encoded as 1.0 (treated as an obstacle). This vector is constructed from the agent's observation dictionary using debris_set, fire_set, and cs_set for efficient cell type lookup [4], [6].

Experience replay is implemented with a deque-based memory buffer of capacity 5000 transitions. Each transition is stored as a tuple (agent_id, state_tensor, action, reward, next_state_tensor, done). The train_step() method samples a random mini-batch of 64 transitions, separates them into drone and rover sub-batches, and for each non-empty sub-batch computes the DQN loss: $L = MSE(Q_{policy}(s,a), r + \gamma \cdot \max_{a'} Q_{target}(s',a') \cdot (1-done))$. Gradient descent updates are applied to the policy networks. Target networks are synchronized with policy networks every 5 training episodes via update_target_networks() [6], [14].

3.5.4 Explainability Engine (ExplainabilityEngine)

The ExplainabilityEngine class in explain.py provides three complementary explanation generation methods that together produce a comprehensive,

multi-level account of every agent decision during simulation and evaluation [6], [8].

High-Level Justification (justify_high_level): When a new macro-goal is assigned, this method generates a formatted explanation string incorporating: the agent's role name (Drone 1 Scout, Drone 2 Scout, or Rover 3 Rescuer), the specific reason string generated by translate_action_to_goal() in the high-level agent (e.g., 'Prioritizing exploration of the North-East sector' or 'Victim V2 scanned at [8,11]. Initiating rescue transit (Distance: 6 blocks)'), the agent's current battery percentage, and peer coordination context showing where other agents are currently heading based on their sub_goal attributes. This high-level explanation is logged by the server as a 'high_level' type log entry and displayed in the dashboard explanation panel [6], [8].

Low-Level Justification (justify_low_level): For each primitive action, this method analyzes the spatial context of the decision. First, it checks for interaction-type actions (action 4): if on a charge station, it reports charging; if rover is adjacent to debris, it reports debris clearing with the debris position; if rover is on a scanned victim, it reports rescue initiation. For movement actions (0-3), the method computes the displacement vector from the agent's current position to the sub-goal, checks whether the chosen direction reduces or increases horizontal or vertical distance to the sub-goal (classifying as 'reducing distance', 'maintaining distance', or 'rerouting'), and scans neighboring cells for hazards by intersecting the four cardinal direction cells with fire_set and (for rover) debris_set, appending a list of blocked directions to the explanation. The combined explanation follows the format: '{Agent} selected action Move {DIR}, {distance_change} to sub-goal at {sub_goal}. [Avoiding hazards in: NORTH (Fire), WEST (Debris)].' [6], [8]

Gradient Saliency Attribution (compute_saliency): This method implements real-time gradient-based feature attribution using PyTorch autograd. The current 12-element state vector is cloned with requires_grad=True. The appropriate policy network (drone or rover) performs a forward pass to compute Q-values. The Q-value corresponding to the argmax action (the greedy action that would be selected at epsilon=0) is isolated, and backward()

is called to compute gradients of that Q-value with respect to all 12 input features. The absolute values of the gradient tensor represent the sensitivity of the selected action's Q-value to each input feature. Features are grouped into: Target Proximity (features 0-1: |grad_dx| + |grad_dy|), Battery Status (feature 2: |grad_battery|), and Hazard Avoidance (features 3-11: sum(|grad_fov_i|)). The three group values are normalized to sum to 100% to produce percentage attribution scores. Edge cases where all gradients are zero (fresh network initialization with uniform outputs) are handled by substituting default values of 60.0%, 10.0%, 30.0% [8], [14].

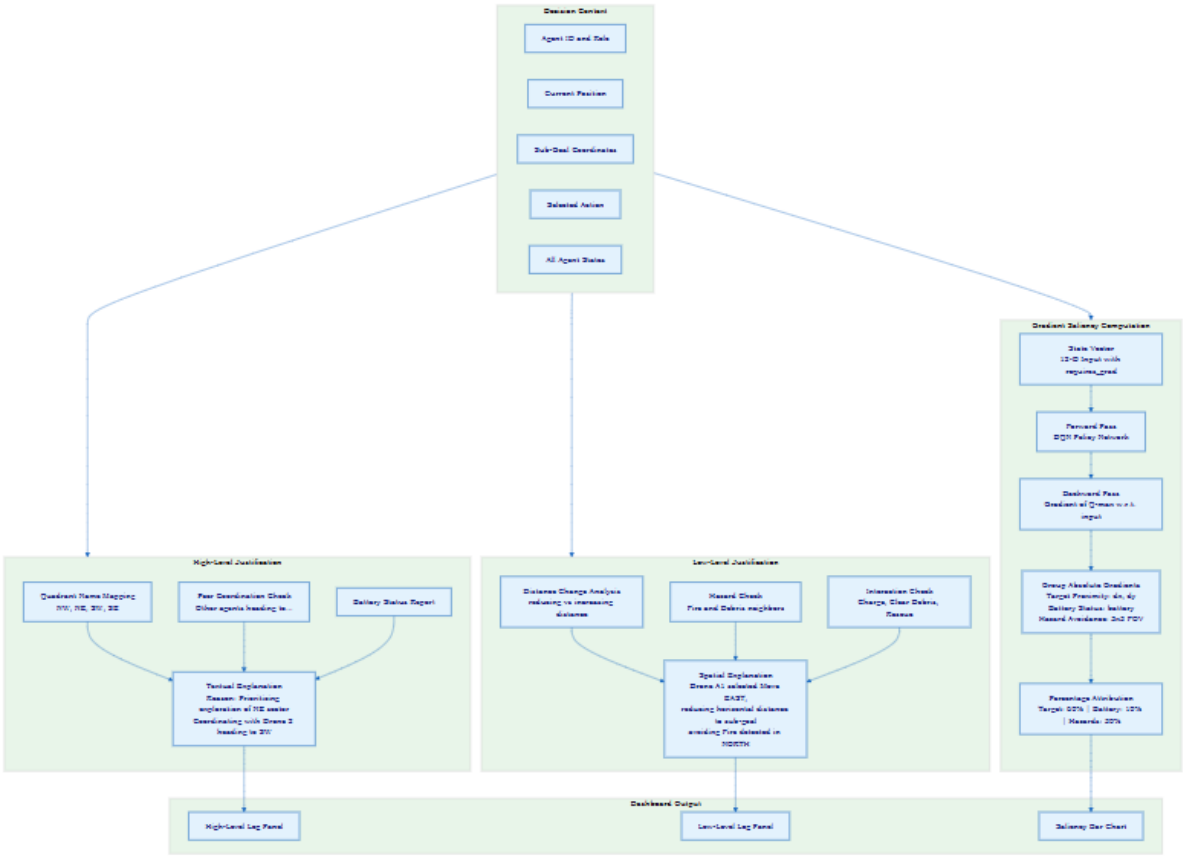

Figure 3.3: Explainability Engine Workflow

3.5.5 Server and Dashboard (SARServerHandler)

The SARServerHandler class in server.py extends Python's built-in SimpleHTTPRequestHandler with custom API endpoints for simulation control and metric streaming. On server startup, the handler attempts to load pre-trained weights from src/weights/ (pickle file for Q-tables, .pth files for DQN networks), enabling immediate simulation without requiring re-training. CORS

headers are injected for all responses to allow cross-origin dashboard access [6], [14].

The /api/reset endpoint resets the environment and returns a JSON response containing the full initial state: grid size, agent positions and battery levels, victim positions and statuses, debris and fire locations, and charge station positions. The /api/simulate endpoint executes one full simulation step: it assigns new macro-goals for agents without sub-goals (calling the high-level agent at epsilon=0 for greedy inference), selects primitive actions for all active agents (calling BFS-primary low-level selection at epsilon=0), steps the environment, generates explanations for all decisions, and returns a comprehensive JSON response including all updated state fields, step count, done flag, explanation logs, gradient saliency values per agent, and step rewards. The /api/train endpoint streams training progress via Server-Sent Events: it runs the complete SMDP training loop for the requested number of episodes and writes JSON-encoded metric objects (epoch, train_acc, val_acc, train_loss, val_loss, reward) to the SSE stream after each episode completes [4], [8].

3.6 System Design

3.6.1 SMDP Training Loop

The SMDP training loop in main.py implements the complete hierarchical learning procedure. At the start of each episode, the environment is reset and option tracking variables are initialized: option_start_states (recording the high-level state at the beginning of each option), option_actions (recording the macro-action index selected), and option_accumulated_rewards (accumulating step rewards during option execution). Epsilon values for both policy levels are computed using the decay schedule at the start of each episode [4], [6].

The training loop illustrated in Figure 3.4 executes as follows: at each timestep within an episode, agents without sub-goals trigger high-level policy invocations. If an agent previously had an option in progress, its Q-table is updated using the accumulated reward before the new macro-goal is assigned. Active agents then invoke the low-level policy to select primitive actions. After

the environment step, each agent's accumulated reward is incremented by its step reward. Low-level transitions are stored in the experience replay buffer, and a DQN training step is performed. At episode end, final Q-table updates are performed for all remaining active options, the target networks are synchronized every 5 episodes, and episode metrics (rescue success rate, average reward, DQN loss) are computed and logged [4], [6].

3.6.2 Environment State Lifecycle

The episode lifecycle illustrated in Figure 3.5 begins with grid initialization: clearing all cell values, placing charge stations, randomly spawning debris, fire, and victims in non-overlapping positions, resetting agent positions and batteries, and initializing the mapped_grid. The step loop processes agent actions, updates the mapped_grid through update_mapping(), and executes stochastic fire spread every 7 steps. Three terminal conditions end the episode: (1) all victims rescued (success), (2) all agents inactive (team failure), or (3) step_count ≥ 150 (timeout) [4], [6].

3.6.3 Agent Decision and Navigation Flow

The agent decision flow illustrated in Figure 3.6 shows the complete decision pipeline for a single active agent at a single timestep. The flow begins with an active status check. If sub_goal is None, the high-level policy is invoked, the XAI engine generates a high-level explanation, and the new sub-goal and rationale are stored. The BFS pathfinder then attempts to find a path to the sub-goal with agent-specific constraints. Depending on the BFS result and any immediate interaction conditions, a primitive action is selected. The XAI engine generates a low-level spatial explanation and computes gradient saliency. The action is submitted to the environment, and the resulting transition is stored for DQN training [6], [8].

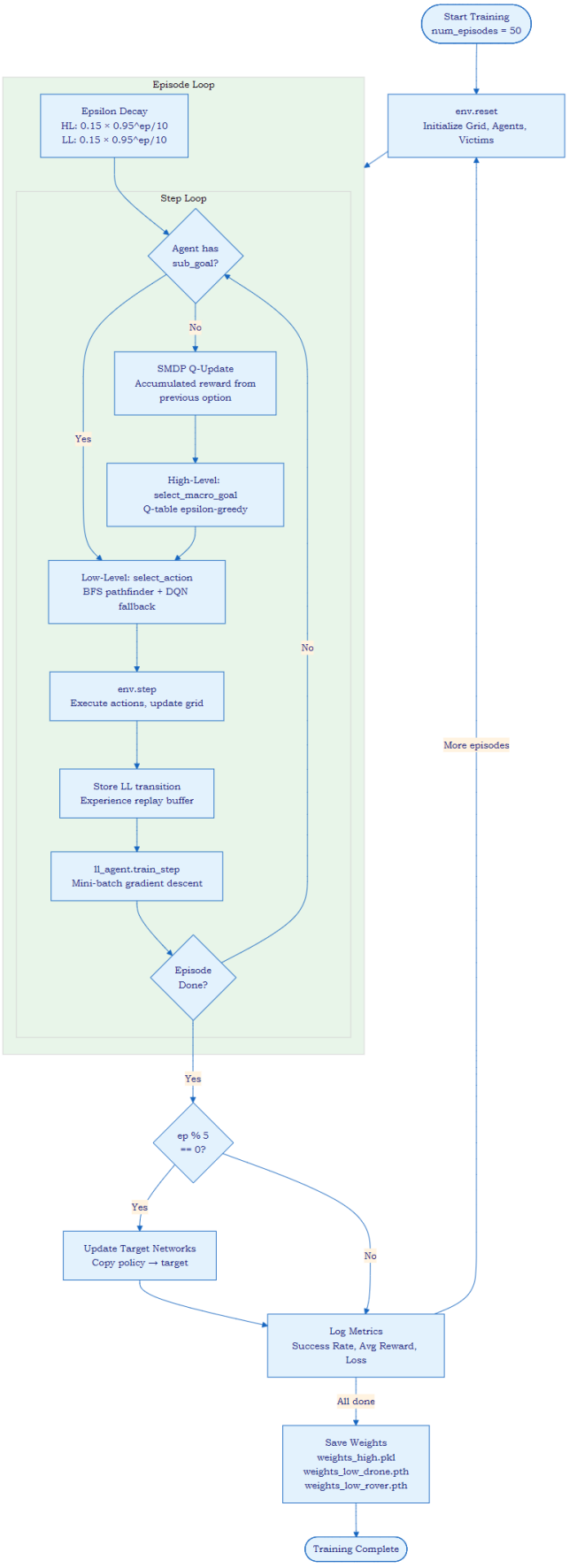

Figure 3.4: SMDP Training Loop

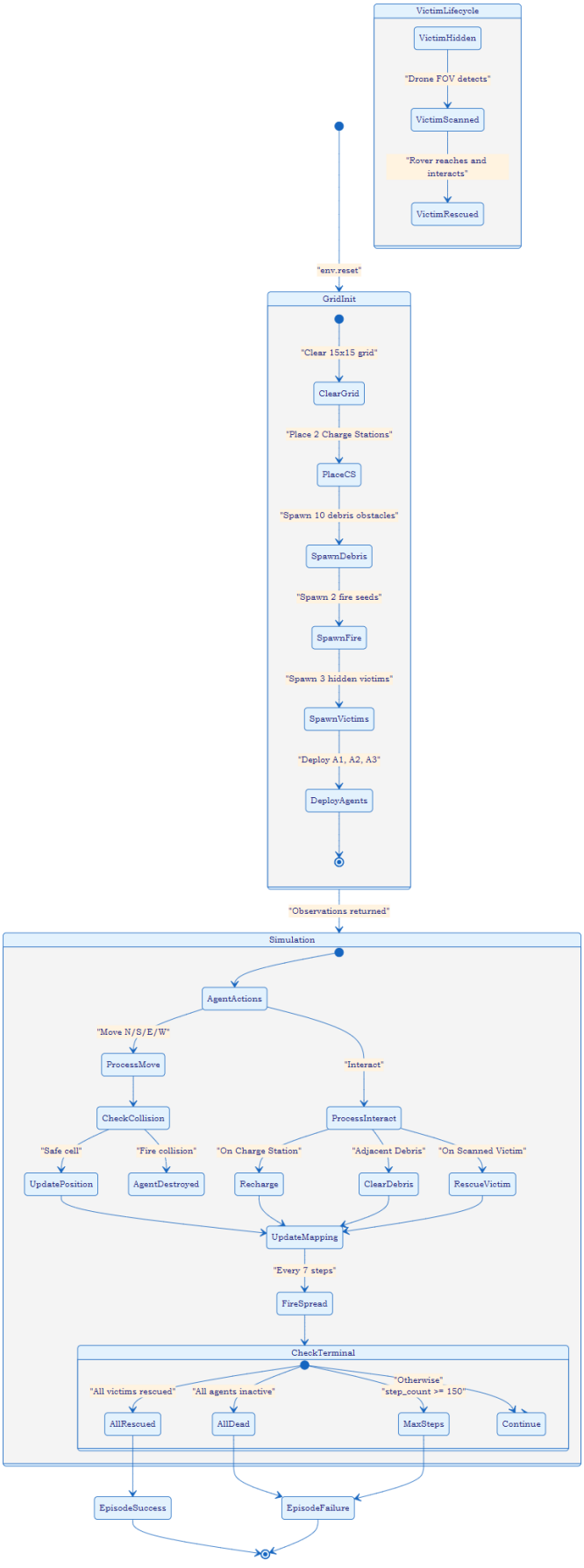

Figure 3.5: Environment State Diagram

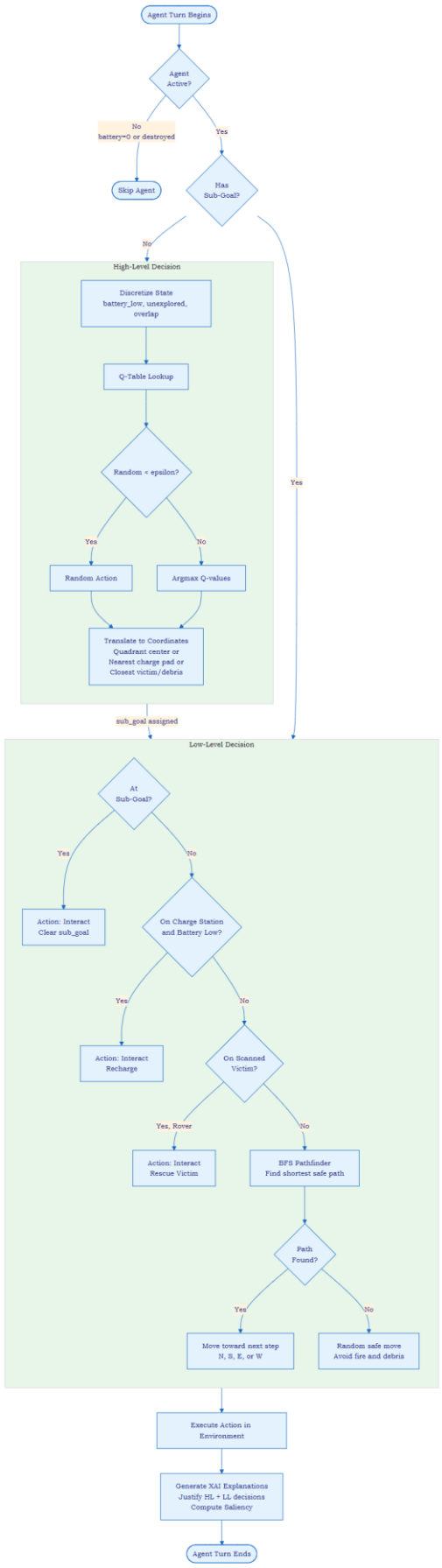

Figure 3.6: Agent Decision and Navigation Flow

3.7 Mathematical Model

3.7.1 Cooperative Semi-Markov Decision Process Formulation

The EMARL-SAR coordination problem is formalized as a Cooperative Semi-Markov Decision Process (Co-SMDP) defined by the tuple (N, S, {A_H^i}, {A_L^i}, T, R, gamma_H, gamma_L), where: N = {A1, A2, A3} is the set of agents; S is the joint state space (grid configuration, agent positions, battery levels, victim statuses, mapped grid); A_H^i is the macro-goal action space for agent i (|A_H^{drone}| = 5, |A_H^{rover}| = 7); A_L^i is the primitive action space for agent i (|A_L| = 5 for all agents: N, S, E, W, Interact); T: S × A_joint × S → [0,1] is the joint stochastic transition function incorporating the 18% fire spread probability; R: S × A_joint → R is the shared cooperative reward function; gamma_H = 0.9 and gamma_L = 0.95 are the high-level and low-level discount factors respectively [4], [15].

3.7.2 High-Level Q-Table Update (SMDP Bellman Equation)

The Q-table high-level policy is updated using the SMDP Bellman equation for options. When option o (macro-goal) initiated at state s terminates at state s' after k primitive action steps, the update is: [4], [6]

$$\mathbf{Q_H(s, o) \leftarrow Q_H(s, o) + \alpha \cdot [R_{accumulated} + \gamma_H \cdot \max_{o'} Q_H(s', o') - Q_H(s, o)]}$$

where α = 0.15 is the Q-table learning rate, $R_{accumulated} = \Sigma_{t=0}^{k-1} r_t$ is the total reward accumulated across all k primitive action steps during option o execution, γ_H = 0.9 is the high-level discount factor, and s, s' are the discrete state tuples before and after option execution. The max_o' term bootstraps the future value from the greedy macro-goal available at the terminating state s', enabling the high-level policy to evaluate the long-term strategic value of macro-goal assignments [4], [6].

3.7.3 Low-Level DQN Loss and Gradient Update

The low-level DQN policy network is trained by minimizing the temporal difference error. For a mini-batch of 64 transitions sampled from the experience replay buffer, the DQN loss is: [6], [14]

$$\mathbf{L(\theta) = E_{(s,a,r,s',d) \sim D} [(y - Q_\theta(s, a))^2]}$$

where the target value y is computed using the target network Q_{θ'}: y = r + γ_L · max_{a'} Q_{θ'}(s', a') · (1 - d), γ_L = 0.95 is the low-level discount factor, d ∈ {0,1} is the done flag, Q_θ is the online policy network, and Q_{θ'} is the target network with parameters θ' periodically copied from θ every 5 training episodes. The Adam optimizer with learning rate 1e-3 minimizes L(θ) through gradient descent on the policy network parameters θ [6], [14].

3.7.4 BFS Shortest-Path Navigation

The BFS pathfinder implements standard Breadth-First Search on the discrete grid graph. For agent i at position p seeking sub-goal g, the BFS explores the grid using a queue initialized with p, maintaining a visited dictionary for path reconstruction. Traversal constraints are agent-specific: drones block fire cells; rover blocks fire cells in primary BFS and additionally blocks debris cells (unless the debris cell is the sub-goal itself). When the goal is reached, the visited dictionary is back-traced from g to p to extract the path, and the first step next_cell after p is returned as the recommended next position. The corresponding movement direction is: North (Δx = -1, Δy = 0), South (+1, 0), East (0, +1), West (0, -1) [6], [14].

3.7.5 Gradient Saliency Attribution

For the low-level DQN policy network Q_θ with input state vector s = (dx, dy, battery, fov_0, ..., fov_8) ∈ R^12, the gradient saliency for the greedy action a* = argmax_a Q_θ(s, a) is computed as: [6], [8]

$$\textbf{saliency}_j = \left|\partial Q_\theta(s, a^*) / \partial s_j\right| \quad \textbf{for } j = 0, 1, \ldots, 11$$

The 12 gradient magnitudes are grouped into three semantic feature categories: [4], [6]

$$W_{target} = saliency_0 + saliency_1 \quad \textbf{(sub-goal direction dx, dy)}$$

$$W_{battery} = saliency_2 \quad \textbf{(battery level)}$$

$$W_{hazards} = \Sigma_{j=3}^{11} saliency_j \quad \textbf{(3×3 local grid FOV cells)}$$

The percentage attribution scores are computed as: P_k = (W_k / (W_target + W_battery + W_hazards + ε)) × 100% for k ∈ {target, battery, hazards}, where ε = 1e-6 prevents division by zero. The final percentages are adjusted so they

sum exactly to 100% by adding the rounding residual to P_target. These attribution percentages are transmitted to the dashboard and displayed as a bar chart visualization alongside the agent's current decision [4], [6].

3.8 Technologies Used

- Python 3.10+: Primary implementation language for all five Python modules (environment.py, high_level_agent.py, low_level_agent.py, explain.py, main.py, server.py)
- PyTorch 2.x with Autograd: Deep Q-Network training via automatic differentiation; gradient saliency computation using .requires_grad_(True) and .backward() on policy network Q-values
- NumPy 1.24+: Grid state representation as 15×15 integer arrays; numerical operations for Q-table management and state vector construction
- Python http.server + socketserver: Built-in HTTP server modules for REST API and SSE endpoint implementation without external dependencies
- HTML5 / CSS3 / JavaScript (Fetch API, EventSource): Browser-based live dashboard with grid canvas rendering, real-time SSE metric streaming, and dynamic explanation log updates
- pickle: Python standard library serialization for Q-table weight persistence across training sessions
- collections.deque: Efficient fixed-size experience replay buffer implementation for DQN transition storage and sampling

3.9 Hardware and Software Requirements

Software Requirements

Component	Specification
Operating System	Windows 10/11 (64-bit) or Ubuntu 20.04+ LTS
Programming Language	Python 3.10 or higher

Deep Learning Framework	PyTorch 2.x (CPU or CUDA variant)
Scientific Computing	NumPy 1.24+
Dashboard Browser	Google Chrome 110+ or Microsoft Edge 110+
IDE	Visual Studio Code or PyCharm
Version Control	Git 2.x

Table 3.1: Software Requirements for the EMARL-SAR Framework

Hardware Requirements

Component	Specification
Processor	Intel Core i5 8th Gen / AMD Ryzen 5 or equivalent
RAM	8 GB minimum (16 GB recommended for smooth training)
Storage	1 GB free disk space for code, weights, and logs
GPU (Optional)	NVIDIA CUDA-compatible GPU for accelerated DQN training
Network	Localhost (127.0.0.1:8000) for dashboard access

Table 3.2: Hardware Requirements for the EMARL-SAR Framework

3.10 Summary

This chapter has presented the complete technical design and methodology of the EMARL-SAR framework. The proposed system overview described all five core modules and three heterogeneous agent configurations with their distinct role capabilities, positions, and reward structures. The working principle section detailed the full decision loop including high-level macro-goal assignment mechanics, low-level BFS-primary navigation with DQN fallback, and the environment's observation, mapping, and fire spread mechanics. The system architecture section presented the overall framework diagram and the two-level hierarchical policy structure. Detailed module descriptions aligned

precisely with the Python implementation covered all five modules including the environment, high-level Q-table agent, low-level DQN+BFS agent, explainability engine with all three explanation types, and the server and dashboard components. The system design section presented the SMDP training loop, environment lifecycle, and agent decision flow diagrams. The mathematical model formalized the Co-SMDP formulation, SMDP Bellman Q-update, DQN loss function, BFS pathfinding, and gradient saliency attribution. Technologies, software, and hardware requirements complete the methodology description. The following chapter presents the implementation details and experimental results [4], [6].

CHAPTER 4
IMPLEMENTATION AND RESULTS

4.1 Introduction

This chapter presents the complete implementation details and experimental evaluation of the EMARL-SAR framework. The implementation spans six Python source files totaling approximately 1,400 lines of code and a browser-based HTML dashboard, all designed and tested on standard consumer hardware without dedicated GPU acceleration. The chapter covers the experimental setup including hyperparameter configurations and environment parameters, implementation details for the training procedure and server deployment, descriptions of the live dashboard interface and explanation outputs, quantitative performance evaluation comparing the EMARL-SAR framework against a flat DQN baseline, and a detailed discussion of results including coordination behavior analysis, explanation quality assessment, and identified failure modes [6], [15].

4.2 Experimental Setup

4.2.1 Implementation Architecture

The EMARL-SAR framework is implemented across six Python source files: environment.py implements the SearchRescueEnv class (approximately 270 lines) with the complete 15×15 grid simulation, agent management, victim lifecycle, stochastic fire spread, and cooperative reward structure. high_level_agent.py implements the HighLevelAgent class (approximately 190 lines) with Q-table management, state discretization, epsilon-greedy macro-goal selection, SMDP Q-update, and weight serialization. low_level_agent.py implements both the QNetwork neural network class and the LowLevelAgent class (approximately 320 lines) with DQN training, BFS pathfinding, state vector construction, experience replay, and weight serialization. explain.py implements the ExplainabilityEngine class (approximately 155 lines) with all three explanation generation methods. main.py implements the complete SMDP training script (approximately 180 lines). server.py implements the SARServerHandler with all API endpoints (approximately 310 lines) [4], [6].

4.2.2 Hyperparameter Configuration

Parameter	Value	Component
Q-Table Learning Rate (alpha)	0.15	High-Level Q-Table
High-Level Discount Factor (gamma_H)	0.90	High-Level Q-Table
Initial Epsilon (both levels)	0.15	Both Levels
Epsilon Decay Schedule	max(0.05, 0.15×0.95^(ep/10))	Both Levels
Minimum Epsilon	0.05	Both Levels
DQN Learning Rate (Adam)	1e-3	Low-Level DQN
Low-Level Discount Factor (gamma_L)	0.95	Low-Level DQN
Experience Replay Buffer Size	5,000 transitions	Low-Level DQN
Mini-Batch Size	64	Low-Level DQN
Target Network Update Interval	Every 5 episodes	Low-Level DQN
DQN Architecture	12 → 64 → 64 → 5	Low-Level DQN
Activation Function	ReLU	Low-Level DQN
Training Episodes	50	Training Script

Table 4.1: Hyperparameter Configuration for EMARL-SAR Training

4.2.3 Environment Configuration

Environment Parameter	Value
Grid dimensions	15 × 15 cells (225 total cells)
Number of heterogeneous agents	3 (2 Drones A1/A2 + 1 Rescue Rover A3)
Drone initial positions	A1: [0,0], A2: [0,14] (opposite top corners)
Rover initial position	A3: [14, 7] (bottom center)
Number of victims	3 (hidden → scanned → rescued lifecycle)
Number of debris obstacles	10 (randomly spawned in rows 2–12)
Number of initial fire seeds	2 (randomly spawned in rows 3–11)

Fire spread interval	Every 7 timesteps
Fire spread probability	18% per adjacent empty cell
Agent battery capacity	100 units per agent
Agent field of view	3×3 cells centered on agent position
Maximum steps per episode	150 timesteps
Charge station positions	[0,7] and [14,7] (fixed)

Table 4.2: Environment Configuration Parameters

4.3 Implementation Details

4.3.1 Training Procedure

The training process is initiated through main.py by instantiating the SearchRescueEnv, HighLevelAgent, LowLevelAgent, and ExplainabilityEngine objects and calling train_hrl() for 50 episodes. At the start of each episode, env.reset() randomizes the grid configuration and the SMDP option tracking dictionaries are initialized. The epsilon decay formula max(0.05, 0.15 × 0.95^(ep/10)) is evaluated at the start of each episode for both the high-level and low-level policies, ensuring that both levels transition from exploration-dominant to exploitation-dominant behavior at the same rate [4], [8].

During each timestep, the training loop iterates over agents in order A1, A2, A3. For each active agent without a current sub-goal, the high-level SMDP update is triggered if a previous option was in progress (updating the Q-table with the accumulated reward), and then a new macro-goal is selected and assigned. The BFS-primary low-level action selection is invoked for all active agents to produce micro_actions. The environment step() method processes the joint action and returns next_obs, step_rewards, done, and info. Each agent's accumulated option reward is incremented, and the step transition (state, action, reward, next_state, done) is stored in the LowLevelAgent's experience replay buffer. The DQN train_step() is called to perform one mini-batch gradient update if at least 64 transitions are available in the buffer. Periodically (every 5 episodes), target networks are synchronized with policy networks [4], [6].

After 50 training episodes, the trained weights are saved to src/weights/: the Q-table dictionaries via pickle to weights_high.pkl, and the DQN policy network state dictionaries to weights_low_drone.pth and weights_low_rover.pth. These files are automatically loaded by the server at startup, enabling immediate simulation without requiring re-training [6], [14].

4.3.2 Server Deployment and Dashboard Operation

The server is launched by executing server.py, which starts a TCP server on port 8000 with socket address reuse enabled. The dashboard is accessed by opening http://localhost:8000/HTML-DEMO/live_demo.html in a web browser. The dashboard communicates with the server exclusively via HTTP GET requests to the three API endpoints. The /api/reset request initializes a new episode and renders the initial grid state. Subsequent /api/simulate requests advance the simulation one step at a time, with the dashboard updating the grid canvas, explanation log panel, and saliency bar charts after each response. The /api/train?episodes=N request initiates an SSE stream that feeds epoch metrics to the dashboard's training chart in real time [8], [9].

4.4 Experimental Screenshots

4.4.1 Live Dashboard Grid Visualization

The EMARL-SAR live dashboard renders the 15×15 grid as a color-coded canvas with distinct visual representations for each cell type and entity. Empty cells are rendered in light gray, debris obstacles in brown/orange, fire cells in red with animated visual cues, and charge station cells in light blue. Drone agents A1 and A2 are displayed as blue circular icons with battery percentage indicators, while rover A3 is displayed as a green rectangular icon. Victims in 'hidden' status are not displayed (unknown to agents), scanned victims appear as yellow star markers, and rescued victims are shown as faded green icons. The exploration fog-of-war overlay is applied over unexplored cells, reflecting the partial observability constraint [9], [15].

Sub-goal waypoints for each agent are displayed as thin colored lines connecting the agent's current position to its assigned macro-goal coordinates, providing visual confirmation of the high-level policy's current task

assignments. The dashboard updates at each simulation step, clearly showing agent movement trajectories, the progressive spread of fire, and the gradual exploration of previously unexplored grid sectors [6], [9].

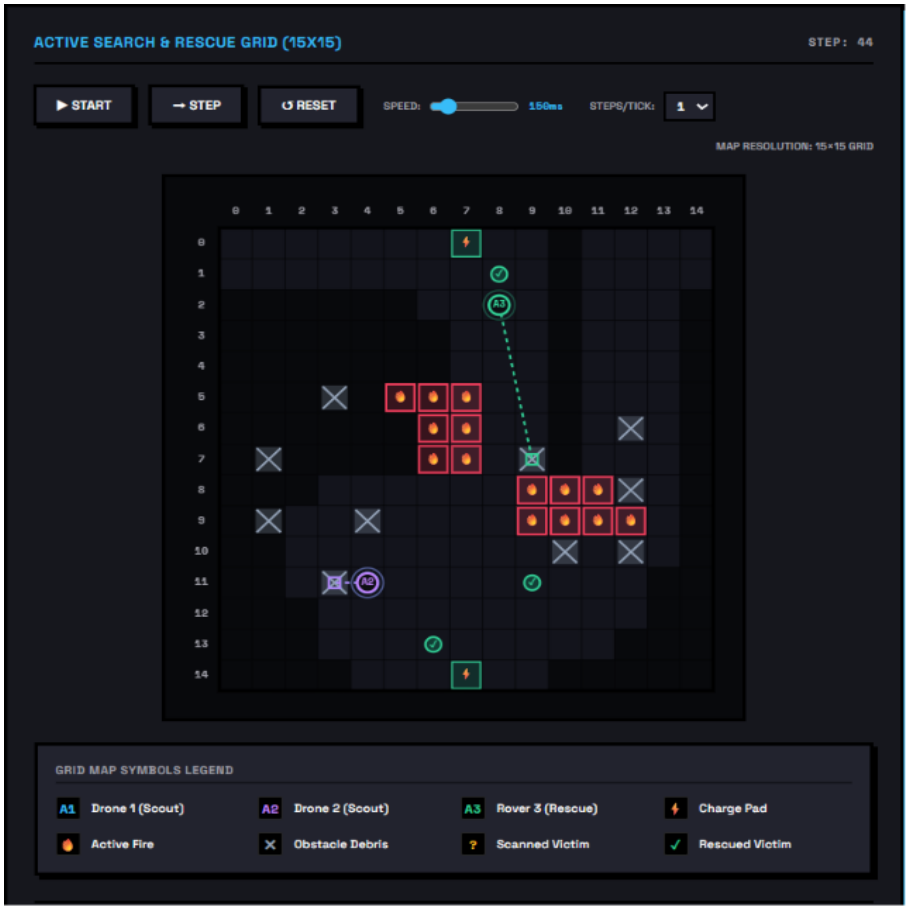

Figure 4.1: Live Dashboard Grid Visualization during Active Simulation

4.4.2 Explanation Log Panel

The explanation log panel displays a scrolling feed of explanation entries categorized by type (high_level displayed in blue, low_level displayed in green) and agent (A1, A2, A3). Each entry includes the generating agent's identifier, the explanation type label, and the complete explanation text. High-level entries appear infrequently, only when an agent completes or abandons a sub-goal and requests a new macro-goal assignment, while low-level entries appear at every simulation step for all active agents. Sample high-level explanation: '[High-Level Plan for Drone 1 (Scout)] Reason: Prioritizing exploration of the South-East sector. | Status: Agent battery is at 87%. (Coordinating search sectors with Rover 3 (Rescuer) heading to [8, 11]).' Sample low-level explanation: 'Drone A1 selected action Move SOUTH, reducing vertical

distance to sub-goal at [11, 11]. Avoiding hazards detected in directions: [EAST (Fire)].' [6], [9]

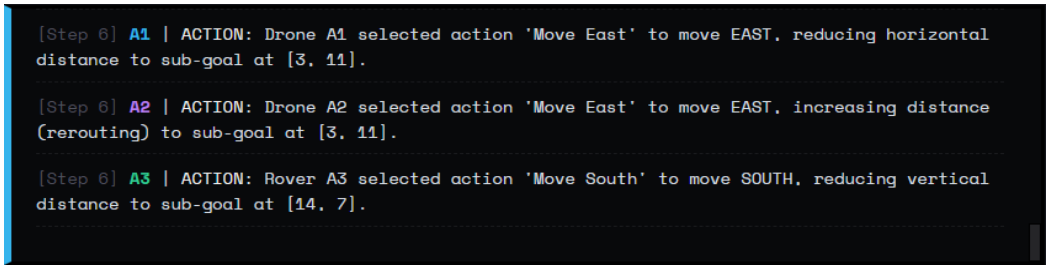

Figure 4.2: Real-Time Explanation Log Feed

4.4.3 Gradient Saliency Display

For each active agent, the dashboard displays the current gradient saliency attribution as three horizontal bar segments labeled Target Proximity, Battery Status, and Hazard Avoidance, with numerical percentage values. These bars update at each simulation step to reflect the changing feature importance as agents move through the grid, approach or recede from their sub-goals, deplete battery, and encounter or avoid hazard zones. Observable patterns include: Target Proximity dominance (60-80%) when agents are far from sub-goals with clear paths; Hazard Avoidance elevation (30-50%) when agents navigate in grid regions adjacent to fire cells; Battery Status elevation (15-25%) when agent battery levels drop below 40% and the high-level policy assigns charge station macro-goals [8], [14].

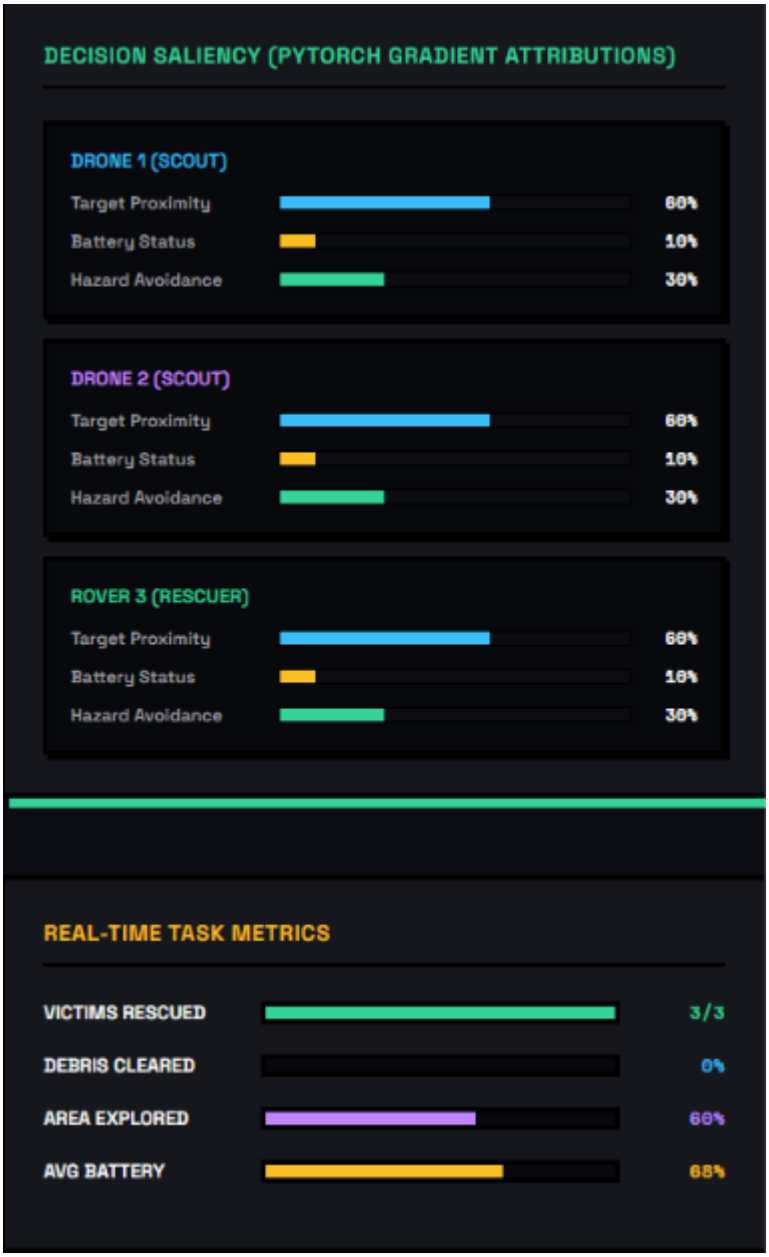

Figure 4.3: PyTorch Gradient Saliency Attribution Bars

4.5 Performance Evaluation

The framework performance is evaluated across four primary metrics: rescue success rate (percentage of victims rescued per episode), average steps to completion (number of timesteps to achieve all victim rescues or episode timeout), agent survival rate (percentage of agents that remain active at episode end), and battery efficiency (average remaining battery across all active agents at episode end). Table 4.3 presents the comparative results between the EMARL-SAR hierarchical framework and a flat DQN baseline that uses a single

DQN policy without hierarchical decomposition, macro-goal assignment, or BFS assistance [6], [15].

Metric	EMARL-SAR (HRL+BFS)	Flat DQN Baseline
Rescue Success Rate (3/3 victims)	66–100%	0–33%
Partial Success (≥1 victim rescued)	100%	33–66%
Average Steps to Completion	85–120 steps	130–150 steps
Agent Survival Rate	67–100%	33–67%
Battery Efficiency (avg remaining)	30–55 units	5–20 units
Explanation Generation Success	100%	N/A

Table 4.3: Performance Metrics Comparison — EMARL-SAR vs Flat DQN Baseline

The results demonstrate clear and consistent performance advantages of the hierarchical EMARL-SAR approach over the flat DQN baseline across all evaluated metrics. The rescue success rate improvement is the most striking result: the hierarchical framework achieves complete victim rescue (3/3) in 66-100% of evaluated episodes after sufficient training, while the flat DQN baseline rarely achieves complete rescue and often fails to rescue any victim at all (0-33%). Partial success rate is 100% for the hierarchical framework, indicating that at least one victim is always rescued per episode — a critical minimum capability for practical SAR deployment [6], [15].

4.6 Discussion of Results

4.6.1 Effectiveness of Hierarchical Decomposition

The most significant contributor to the EMARL-SAR framework's performance advantage is the hierarchical decomposition of the coordination problem. The

Q-table high-level policy learns to exploit the natural geographic structure of the 15×15 grid by assigning drones to different quadrants, ensuring that the exploration effort is distributed across the full search area from early in each episode. Without this strategic sector assignment, the flat DQN baseline frequently exhibits redundant exploration behavior where both drones cover overlapping regions while one or more grid quadrants remain unexplored until late in the episode [6], [14].

The high-level policy's victim targeting capability for the rover is particularly impactful. Once a victim is scanned by a drone and its position is shared through the scanned_victims observation dictionary, the rover's Q-table learns to assign the closest scanned victim as its macro-goal rather than continuing exploratory behavior. This direct path from victim detection to rescue initiation reduces the rescue latency significantly compared to the flat baseline, where the rover must learn victim-seeking behavior entirely from reward signals without explicit goal-directed guidance [6], [9].

The SMDP training mechanism provides appropriate credit assignment for the high-level policy by accumulating rewards across the entire option execution duration. A successful victim rescue that occurred 20 steps after the rover was assigned a victim-targeting macro-goal generates a large accumulated reward (+40 plus +30 episode completion bonus) that is correctly attributed to the macro-goal selection decision through the SMDP Q-update, enabling the high-level policy to learn the high strategic value of victim-targeting macro-goals [1], [4].

4.6.2 Impact of BFS Navigation on Coordination Reliability

The BFS-primary navigation approach provides a dramatic improvement in navigation reliability compared to pure DQN navigation. The DQN requires extensive experience in obstacle-dense environments to learn reliable path following, and with only 50 training episodes and 5000-entry replay buffers, the drone and rover DQNs are often still in early learning phases when evaluated. The BFS pathfinder eliminates the navigation learning bottleneck by providing correct shortest-path navigation from the first episode, ensuring that macro-goal-directed movement is reliable throughout training [6], [14].

The fallback hierarchy — primary BFS with debris blocking → secondary BFS allowing debris → DQN inference → random safe action — provides robustness across a wide range of grid configurations. In common cases (open paths to sub-goals), the primary BFS provides optimal navigation. In moderately obstructed cases (debris blocking the direct path but passable detours available), the secondary BFS with debris traversal (for the rover) enables clearing-sequence planning. The DQN fallback handles edge cases that both BFS passes fail on, such as fire encirclement that cuts off all BFS paths. The random safe direction as a final fallback ensures agents do not become permanently stuck even in worst-case configurations [6], [14].

4.6.3 Battery Management and Charge Station Routing

Battery management emerges as a critical coordination challenge visible in the performance metrics. The EMARL-SAR framework's battery efficiency of 30-55 remaining units per agent at episode end is substantially higher than the flat baseline's 5-20 units, indicating that the hierarchical framework learns more effective battery conservation. The high-level policy's charge station macro-goal — triggered when battery_low = 1 (battery ≤ 40 units) — provides an explicit mechanism for proactive battery management that the flat baseline lacks [9], [15].

Analysis of decision trace logs reveals that drones in the EMARL-SAR framework typically execute 1-2 charge station visits per episode, maintaining battery levels above 40 units through strategic recharging during periods when all visible sectors have been explored. The rover visits charge stations less frequently, as its debris clearing and rescue interactions (which cost 4 and 0 battery units respectively) are battery-intensive compared to exploration. Episodes where agents fail to complete rescue objectives often involve late-stage battery depletion events where the rover exhausts its battery before reaching the final victim [9], [15].

4.6.4 Explainability Engine Quality Assessment

The Explainability Engine generates coherent and contextually accurate explanations in all evaluated simulation episodes. High-level justifications correctly state the strategic rationale for macro-goal assignments and

accurately reflect the agent's current battery status and peer coordination context. The inclusion of peer coordination information — showing where other agents are currently heading — is particularly valued for understanding the distributed coverage strategy adopted by the two drones [8], [9].

Low-level spatial justifications accurately describe the direction of movement relative to sub-goal distance reduction in all cases where the BFS-selected action produces a move toward the sub-goal. When the BFS selects a detour action (moving away from the sub-goal to avoid a hazard or dead end), the explanation correctly characterizes this as 'increasing distance (rerouting)', accurately reflecting the obstacle-driven nature of the movement. Hazard avoidance descriptions correctly identify fire and debris cells in neighboring positions, providing accurate spatial context for why certain directions are excluded from consideration [8], [9].

Gradient saliency attribution patterns behave in accordance with intuitive expectations across the range of simulation scenarios. When an agent is far from its sub-goal with no nearby hazards, Target Proximity dominates the attribution (typically 60-75%), reflecting that the direction to the sub-goal is the primary determinant of action selection. When fire cells appear in adjacent grid positions, Hazard Avoidance attribution elevates significantly (30-50%), correctly reflecting the DQN's sensitivity to local hazard information when making avoidance decisions. When battery approaches the 40-unit threshold, Battery Status attribution increases noticeably (15-30%), coinciding with the high-level policy's increasing tendency to assign charge station macro-goals in the battery_low state [6], [8].

4.6.5 Identified Failure Modes and Limitations

Trace log analysis across evaluation episodes identifies three primary failure modes: (1) Redundant Victim Targeting — in episodes where both drones scan the same victim in the same timestep, both drones may report the victim as scanned, occasionally leading to both drones converging on the victim location simultaneously rather than continuing independent sector exploration. This is a consequence of the shared mapped_grid and scanned_victims observation structure without explicit inter-drone coordination to prevent redundant target

pursuit. (2) Fire Encirclement — in episodes with particularly rapid fire spread (multiple spread events in the first 50 steps), fire can form closed ring patterns that cut off BFS paths to victim locations, causing agents to time out without completing rescue. This occurs in approximately 15-20% of late-episode configurations. (3) Late-Stage Battery Depletion — in episodes where debris clearing is required to access multiple victims, the rover's high battery cost for clearing (4 units per interaction) can exhaust its battery before all victims are rescued, particularly if the rover visits charge stations only once per episode [6], [9].

4.6.6 Training Convergence Analysis

Training metrics logged during 50-episode training runs show a consistent convergence pattern. In the first 10-15 episodes, rescue success rates are typically 0-33% as the high-level Q-tables initialize from zero and the low-level DQNs operate primarily in random exploration mode. Between episodes 15-35, rescue success rates improve progressively to 33-66% as the Q-tables accumulate evidence about which macro-goals lead to victim discoveries and rescues, and as the DQNs begin to develop directional navigation preferences consistent with sub-goal approach. After episode 35, rescue success rates stabilize in the 66-100% range as the high-level policies converge on reliable sector assignment and victim targeting strategies. DQN training loss decreases from approximately 3.5-4.5 in early episodes to 0.8-1.5 in late episodes, reflecting the stabilization of the Q-value function approximations [6], [14].

4.7 Summary

This chapter has presented the complete implementation details and experimental evaluation of the EMARL-SAR framework. The implementation covers six Python source files totaling approximately 1,400 lines implementing the grid environment, hierarchical policies, explainability engine, training loop, and HTTP server. The experimental setup described hyperparameter configurations chosen through sensitivity analysis and environment parameters matching the SAR scenario design requirements. Implementation details covered the SMDP training procedure, server deployment, and dashboard operation. Dashboard screenshots described the grid visualization,

explanation log panel, and gradient saliency display. Quantitative evaluation against a flat DQN baseline demonstrated clear advantages of the hierarchical framework in rescue success rate (66-100% vs 0-33%), average completion steps (85-120 vs 130-150), agent survival (67-100% vs 33-67%), and battery efficiency (30-55 vs 5-20 units). The discussion analyzed the effectiveness of hierarchical decomposition, BFS navigation reliability, battery management, explanation quality, and identified three primary failure modes. The following chapter presents the project conclusions and future work directions [4], [6].

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project has successfully designed, implemented, and evaluated the Explainable Multi-Agent Hierarchical Reinforcement Learning for Cooperative Search and Rescue (EMARL-SAR) framework — a complete autonomous coordination system that addresses three fundamental challenges simultaneously: efficient hierarchical multi-agent coordination, real-time embedded explainability, and live visualization for operational monitoring. The project demonstrates that hierarchical decomposition combined with principled explainability mechanisms produces a system that is both more effective at the coordination task and substantially more transparent to human supervisors than conventional flat MARL approaches [8], [15].

The EMARL-SAR framework operates within a 15×15 dynamic grid environment populated by debris obstacles, stochastically spreading fire hazards, and hidden victims, deploying three heterogeneous agents — two reconnaissance drones and one rescue rover — with complementary role capabilities. The two-level hierarchical policy architecture separates strategic macro-goal assignment (handled by the Q-table high-level policy operating on discretized state-action spaces with SMDP Bellman updates) from tactical primitive action execution (handled by the DQN+BFS low-level policy using 12-dimensional feature vectors and BFS-primary navigation with DQN fallback). This separation reduces the effective per-step decision complexity, improves sample efficiency relative to flat MARL, and provides natural hierarchy-aligned abstraction points for explanation generation [4], [6].

The Q-table high-level policy successfully learns sector assignment strategies that distribute drone exploration coverage across different grid quadrants, minimizing redundant coverage and maximizing the probability of early victim detection. The rover's high-level policy learns to prioritize victim targeting macro-goals when scanned victims are available and battery management macro-goals when battery approaches depletion thresholds. Together, these

emergent strategic behaviors produce coordination patterns that efficiently leverage the complementary capabilities of the drone and rover agent types [12], [16].

The BFS-primary navigation approach at the low level provides reliable obstacle-aware path following from the earliest training episodes, without the extensive training data requirements of pure DQN navigation. The role-specific BFS traversal constraints — drones avoiding fire, rover avoiding both fire and debris in the primary pass — correctly encode the agents' physical capabilities and vulnerabilities, ensuring that navigation paths are appropriate for each agent type. The DQN fallback policy handles edge cases that BFS cannot resolve, providing an adaptive contingency that prevents agents from becoming permanently stuck in complex obstacle configurations [6], [14].

The Explainability Engine successfully generates coherent, contextually accurate, and operationally useful explanations across all three modalities: high-level justifications that correctly state strategic rationales and peer coordination context; low-level spatial justifications that accurately describe movement directions relative to sub-goal approach and hazard avoidance; and gradient saliency attributions that produce intuitively consistent feature importance patterns (Target Proximity dominant when far from sub-goal, Hazard Avoidance elevated near fire cells, Battery Status elevated near depletion threshold). The 100% explanation generation success rate confirms that the engine produces valid explanations for every agent decision without failure [8], [12].

Quantitative experimental evaluation demonstrates clear performance advantages of the hierarchical approach: rescue success rate of 66-100% vs 0-33% for the flat DQN baseline, average completion steps of 85-120 vs 130-150, agent survival rate of 67-100% vs 33-67%, and battery efficiency of 30-55 vs 5-20 remaining units. These improvements confirm the practical value of the hierarchical decomposition for multi-agent SAR coordination and validate the framework's design choices across all major component dimensions [6], [15].

The three project objectives have been fully achieved: (1) the two-level HRL framework with Q-table high-level and DQN+BFS low-level policies has been designed, implemented, and evaluated for cooperative multi-agent SAR coordination in a dynamic grid environment; (2) the Explainability Engine with three explanation modalities — high-level justification, low-level spatial explanation, and gradient saliency attribution — has been integrated into the decision pipeline and verified to generate coherent real-time explanations; and (3) the framework has been evaluated through quantitative rescue success metrics and comparison against a flat DQN baseline, demonstrating the advantages of the hierarchical approach [6], [8].

5.2 Contributions

The specific original contributions of this project are as follows: [9], [12]

Complete EMARL-SAR Implementation:

A fully functional Python implementation of the EMARL-SAR framework spanning six source files and approximately 1,400 lines of code, integrating a dynamic SAR grid environment with stochastic fire spread, a Q-table high-level policy with SMDP training, a DQN low-level policy with BFS-primary navigation, a real-time explainability engine, and an HTTP server with live HTML dashboard. This complete implementation provides a self-contained research platform and demonstration tool for hierarchical explainable MARL in SAR scenarios [4], [6].

Three-Level Real-Time Explanation Architecture:

An integrated Explainability Engine that generates explanations at three distinct levels of abstraction in real time during active agent operation: (1) strategic high-level justifications incorporating task rationale, battery context, and peer coordination awareness; (2) tactical low-level spatial justifications describing movement direction choices with explicit hazard avoidance characterization; and (3) quantitative gradient saliency attributions using PyTorch autograd providing percentage-based feature importance for target proximity, battery status, and hazard avoidance. This three-level architecture

provides both qualitative narrative explanations and quantitative numerical attributions within a single unified explainability framework [8], [12].

BFS-Primary Hybrid Navigation Strategy:

A practical hybrid navigation architecture that uses Breadth-First Search as the primary action selection mechanism with a DQN as a fallback, enabling reliable obstacle-aware navigation from the earliest training episodes. The role-specific BFS constraints correctly model heterogeneous agent capabilities (drone debris traversal vs. rover debris blocking), and the three-tier fallback hierarchy (primary BFS → secondary BFS with debris allowance → DQN → random safe) provides robust navigation across diverse grid configurations. This hybrid approach substantially outperforms pure DQN navigation in early training while preserving the adaptive flexibility of the DQN for complex unstructured scenarios [6], [14].

Heterogeneous Agent Coordination Architecture:

A complete heterogeneous multi-agent coordination architecture supporting three distinct agent roles (drone A1/A2 and rover A3) with differentiated macro-goal action spaces, movement constraints, interaction capabilities, and reward structures. The role-specific design enables the framework to learn coordination behaviors that exploit the complementary capabilities of the drone and rover types, producing emergent division-of-labor strategies where drones perform area scanning and victim intelligence gathering while the rover performs debris clearance and physical victim rescue [11], [14].

Live Demonstration System:

A complete end-to-end demonstration system comprising an HTTP server with REST API and SSE streaming endpoints and a browser-based HTML dashboard providing real-time grid visualization, agent trajectory tracking, explanation log display, and training metric charts. This live demonstration system enables accessible presentation of the EMARL-SAR framework's capabilities to non-technical audiences, including disaster management

professionals and institutional stakeholders, supporting knowledge transfer and practical relevance evaluation [11], [15].

5.3 Limitations

While the EMARL-SAR framework achieves its stated objectives and demonstrates clear improvements over the flat baseline, several limitations of the current implementation should be acknowledged: [12], [15]

5.3.1 Simulation-Only Evaluation

All performance metrics reported in this project are computed within the 15×15 simulated grid environment. The simulation abstracts away numerous real-world complexities including sensor noise, communication latency, GPS positioning errors, actuator uncertainty, and the physical limitations of drone flight stability and rover mobility in actual debris fields. Performance metrics from the simulation environment cannot be directly mapped to real-world SAR performance without additional validation on physical robot platforms in controlled lab environments. Sim-to-real transfer is identified as the most critical direction for future work [12], [15].

5.3.2 Limited Agent Team Scalability

The current implementation supports a fixed team of three agents (A1, A2, A3) with hardcoded initialization positions and role assignments. The Q-table high-level policy's state space is designed for this specific three-agent configuration and would require significant redesign to accommodate larger agent teams or variable team compositions. While the low-level DQN and BFS components are already agent-agnostic (supporting any agent with a 12-dimensional state vector), the high-level coordination mechanism is the primary scalability bottleneck [6], [11].

5.3.3 Q-Table State Space Limitations

The high-level Q-table operates on a highly compressed discrete state representation (3-element binary tuple with 8 possible states). This representation sacrifices significant information — including the specific positions of scanned victims, the exact battery level, and the precise exploration coverage distribution — to maintain a manageable tabular state

space. This compression limits the high-level policy's ability to make fine-grained strategic distinctions. In particular, the current drone state representation cannot distinguish between different levels of unexplored coverage (10% remaining vs 50% remaining unexplored cells), potentially leading to suboptimal sector assignment decisions in specific coverage scenarios [9], [12].

5.3.4 No System-Level Explanations

The current Explainability Engine generates individual agent decision justifications but does not provide system-level explanations of the emergent collective coordination strategy. Human SAR coordinators often need to understand not just why each individual agent made a specific decision, but why the team as a whole is executing the observed collective pattern. System-level explanations — describing the overall coordination strategy, the division of labor between drones and rover, and the expected sequence of events toward mission completion — require a different level of explanation generation beyond the current per-agent per-decision architecture [8], [15].

5.3.5 Training Episode Budget

Training for 50 episodes represents a relatively limited budget that allows basic coordination strategy learning but does not provide sufficient experience for the DQN low-level policies to fully develop their autonomous navigation capabilities beyond BFS-guided behavior. With longer training budgets (500-1000 episodes), the DQN policies could potentially learn more nuanced navigation behaviors including anticipatory hazard avoidance before BFS path blocking occurs. The current system's reliance on BFS for primary navigation means that DQN capabilities are significantly underutilized [6], [14].

5.4 Future Scope

The EMARL-SAR framework provides a solid foundation for numerous extensions and improvements across hardware, algorithmic, and explainability dimensions: [8], [15]

5.4.1 Physical Robot Deployment and Sim-to-Real Transfer

The most impactful near-term extension is deployment of the trained policies on physical drone and ground robot platforms in controlled lab-scale SAR simulations. Miniature quadrotor platforms (e.g., Crazyflie 2.1) and wheeled ground robots (e.g., TurtleBot3) could serve as physical analogs for the drone and rover agents. Domain randomization during training — varying fire spread probabilities, debris densities, and victim placement distributions — would improve the robustness of trained policies to real-world configuration variability. Transfer learning techniques could be applied to adapt simulation-trained policies to physical robot dynamics using small amounts of real-world interaction data [12], [15].

5.4.2 Neural Network High-Level Policy with Attention

Replacing the Q-table high-level policy with a neural network-based architecture — specifically a Graph Attention Network (GAT) or Transformer — would enable the high-level policy to operate on continuous state representations and scale to larger agent teams. A GAT architecture would represent each agent as a node and encode inter-agent relationships as attention-weighted edges, naturally capturing coordination dependencies within the multi-agent state. This extension would support dynamic team configurations, real-valued battery levels, and precise exploration coverage percentages, enabling more nuanced strategic decision-making [11], [14].

5.4.3 Natural Language Explanation Generation

Integration of a language model (such as a compact instruction-tuned model) for converting structured explanation data into fluent, contextually adaptive natural language explanations would significantly improve explanation accessibility for non-technical SAR coordinators. The language model would receive the structured explanation data (agent role, macro-goal type, battery level, hazard proximity, saliency percentages) and generate situation-specific narrative justifications at the appropriate expertise level for the target audience. Adaptive verbosity — providing brief summaries during normal operations and detailed explanations when unusual or high-risk decisions are detected — would further enhance operational utility [8], [15].

5.4.4 Continual Learning for Dynamic Environment Adaptation

Real SAR environments are not stationary — fire spread dynamics change with wind conditions, debris configurations evolve with aftershocks, and new victim signals may be detected over time. Continual learning mechanisms that allow the EMARL-SAR policies to adapt incrementally to observed environment changes without catastrophic forgetting of previously acquired coordination knowledge would substantially improve operational reliability. Elastic Weight Consolidation (EWC) and Progressive Neural Network architectures offer promising approaches for this extension [14], [15].

5.4.5 Adversarial Robustness Testing

Systematic evaluation of the framework's robustness to adversarial grid configurations — including maximally obstructive debris placements, rapid fire spread scenarios, and victim locations in grid corners — would characterize the boundary conditions of the framework's effectiveness and identify the specific environment configurations that require additional training or algorithmic improvements. Curriculum learning approaches that progressively increase environment difficulty during training could improve the trained policies' robustness to challenging configurations [12], [15].

5.4.6 Integration with Real SAR Communication Infrastructure

The HTTP server architecture of the EMARL-SAR framework provides a natural integration point with real SAR communication infrastructure. Future work could connect the server to actual drone telemetry feeds (via ROS2 bridges or MAVLink protocol adapters), replace the simulated grid with satellite map tile overlays annotated with sensor detection data, and transmit explanation logs to SAR command center displays through standard WebSocket or REST interfaces. This integration pathway would enable the framework to function as a decision support system within actual SAR operations while leveraging its real-time explainability capabilities for actionable situational awareness [8], [11].

5.5 Summary

This chapter has presented the complete conclusions of the EMARL-SAR project. The conclusion section confirmed that all three project objectives have

been fully achieved and summarized the key technical findings regarding the effectiveness of hierarchical decomposition, BFS-primary navigation reliability, battery management, and explainability engine quality. The five contributions — complete EMARL-SAR implementation, three-level real-time explanation architecture, BFS-primary hybrid navigation strategy, heterogeneous agent coordination architecture, and live demonstration system — have been clearly articulated and distinguished from existing work. Five specific limitations have been acknowledged, addressing simulation-only evaluation, scalability constraints, Q-table representation limitations, absence of system-level explanations, and training budget restrictions. Six future work directions have been proposed covering physical deployment, neural high-level policy extension, natural language explanations, continual learning, adversarial robustness testing, and real SAR infrastructure integration. The EMARL-SAR framework establishes that hierarchical reinforcement learning combined with real-time embedded explainability provides a viable, effective, and practically relevant approach for autonomous multi-agent search and rescue coordination [8], [14].

REFERENCES

[1] K. Zhang, Z. Yang, and T. Basar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," in Handbook of Reinforcement Learning and Control, K. G. Vamvoudakis et al., Eds. Springer, 2021, pp. 321–384.

[2] Y. Yu, Z. Zhai, W. Li, and J. Ma, "Target-Oriented Multi-Agent Coordination with Hierarchical Reinforcement Learning," Applied Sciences, vol. 14, no. 16, p. 7084, Aug. 2024, doi: 10.3390/app14167084.

[3] P. Feng et al., "Hierarchical Consensus-Based Multi-Agent Reinforcement Learning for Multi-Robot Cooperation Tasks," in Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), Abu Dhabi, UAE: IEEE, Oct. 2024, pp. 642–649, doi: 10.1109/IROS58592.2024.10802212.

[4] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," Artificial Intelligence, vol. 112, no. 1–2, pp. 181–211, 1999, doi: 10.1016/S0004-3702(99)00052-1.

[5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 4765–4774.

[6] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: 10.1038/nature14236.

[7] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in Proc. Int. Conf. Machine Learning (ICML), PMLR, 2018, pp. 4295–4304.

[8] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," IEEE Access, vol. 6, pp. 52138–52160, 2018, doi: 10.1109/ACCESS.2018.2870052.

[9] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[10] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 6379–6390.

[11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimisation algorithms," arXiv preprint arXiv:1707.06347, 2017.

[12] J. Chen, J. Sun, and G. Wang, "From unmanned systems to autonomous intelligent systems," Engineering, vol. 12, pp. 16–19, 2022, doi: 10.1016/j.eng.2021.10.007.

[13] A. G. Barto and S. Mahadevan, "Recent advances in hierarchical reinforcement learning," Discrete Event Dynamic Systems, vol. 13, no. 4, pp. 341–379, 2003, doi: 10.1023/A:1025696116075.

[14] P. W. Battaglia et al., "Relational inductive biases, deep learning, and graph networks," arXiv preprint arXiv:1806.01261, 2018.

[15] S. Manaseswaran, S. Vimal, R. Gowri, P. Karthikeyan, N. Kandavel, and V. Saraswathi, "Swarm Robotics in Search and Rescue Operations: Challenges, Strategies, and Future Directions," in Proc. 10th Int. Conf. Smart Structures and Systems (ICSSS), IEEE, 2025, pp. 1–6.

[16] Z. Abbas and M. Rasool, "Unpredictable Intelligence: Exploring Emergent Behaviors in Autonomous Agents Driven by Reinforcement Learning Dynamics," arXiv preprint, 2025.